
# Agent Memory

Yusen Zhang
yz5296@columbia.edu

# Outline

## Part 1. What Is Agent Memory?

## Part 2. How to Benchmark Agent Memory?

- LoCoMo [1]
- LongMemEval [2]
- MemoryArena [3]
- **VibeCodingMem (ours)**

## Part 3. How Are the Agent Memory Frameworks Designed?

- A-Mem [4]
- Mem0 [5]
- LightMem [6]
- **AIM (ours)**

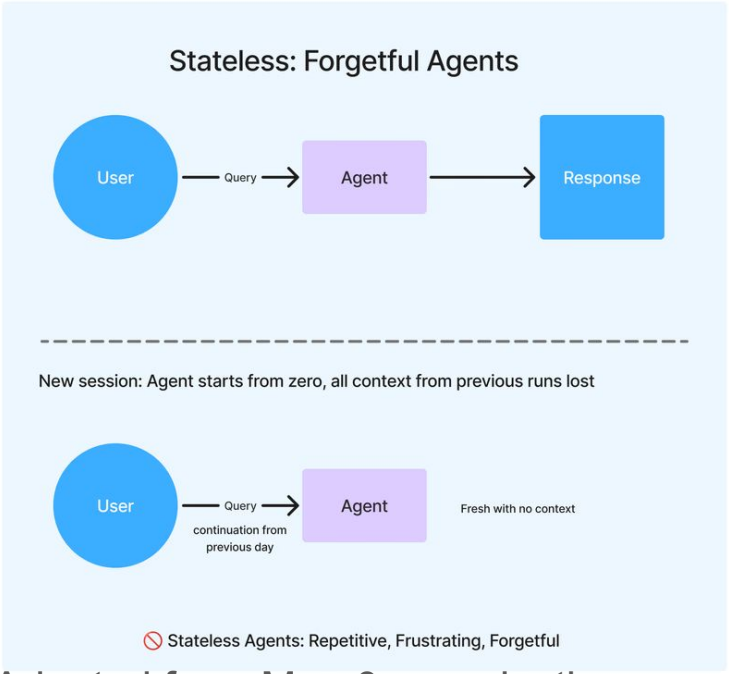
[1] Maharana, Adyasha, et al. "Evaluating very long-term conversational memory of llm agents." ACL 2024. [2] Wu, Di, et al. "Longmemeval: Benchmarking chat assistants on long-term interactive memory." ICLR 2025. [3] He, Zexue, et al. "MemoryArena: Benchmarking agent memory in interdependent multi-session agentic tasks." ArXiv Feb. 2026. [4] Xu, Wujiang, et al. "A-mem: Agentic memory for llm agents." NeuRIPS 2025. [5] Chhikara, Prateek, et al. "Mem0: Building production-ready ai agents with scalable long-term memory." ArXiv Apr. 2025. [6] Fang, Jizhan, et al. "Lightmem: Lightweight and efficient memory-augmented generation." ICLR 2026

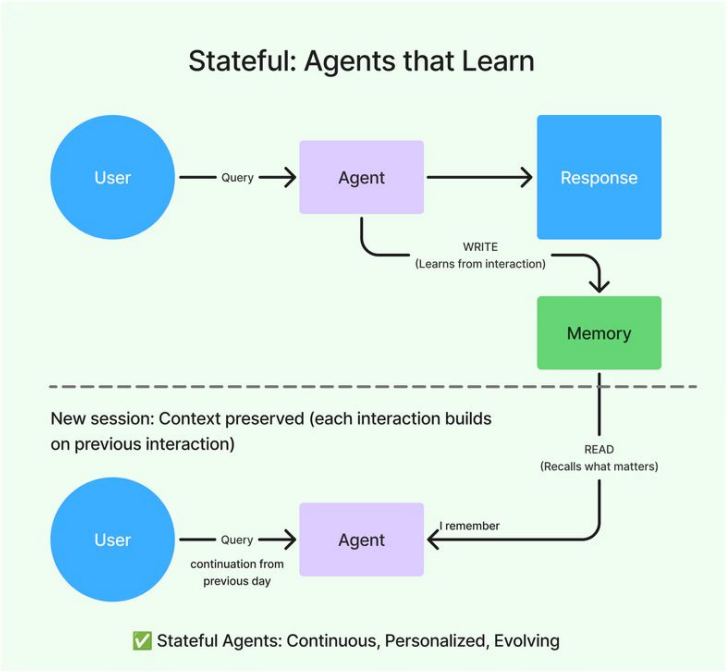
# Part 1. What Is Agent Memory?

# Memory: The Missing Layer in AI Agent Architecture

mem0

From stateless interactions to continuous intelligence




*Adapted from Mem0 organization

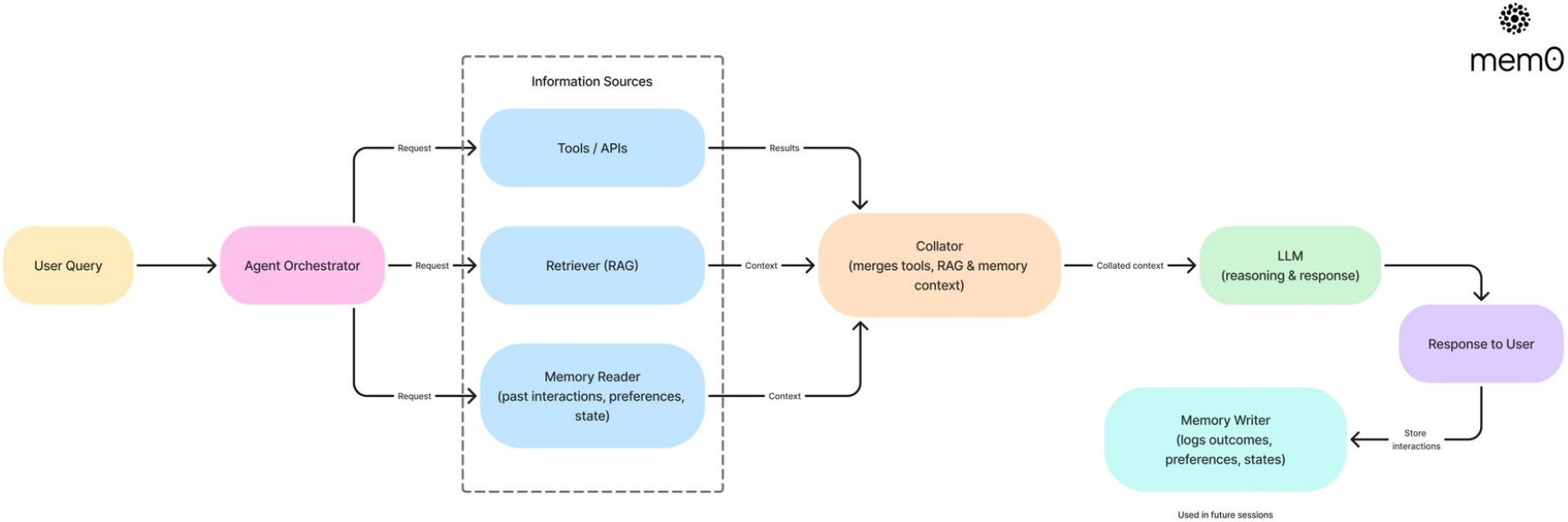
# Agent Memory: Overview

What is the target of Agent Memory?

- In the context of AI agents, memory is the ability to retain and recall relevant information across time, tasks, and multiple user interactions.

Why do we need this?

- It allows agents to remember what happened in the past and use that information to improve behavior in the future.


mem0
Information Sources
User Query
Agent Orchestrator
Request
Request
Request
Tools / APIs
Retriever (RAG)
Memory Reader
(past interactions, preferences, state)
Results
Context
Context
Collator
(merges tools, RAG & memory context)
Collated context
LLM
(reasoning & response)
Response to User
Store interactions
Memory Writer
(logs outcomes, preferences, states)
Used in future sessions

# Context Window ≠ Memory

A common misconception is that large context windows will eliminate the need for memory.

- Calling an LLM with more context is they can be expensive: *more tokens = higher cost and latency*
- Context windows help agents stay consistent *within a session*. Memory allows agents to be intelligent *across sessions*.
- Even with context lengths reaching 100K tokens, the *absence of persistence, prioritization, and salience* makes it insufficient for true intelligence.

# Context Window ≠ Memory

| Feature | Context Window | Memory |
|---|---|---|
| **Retention** | Temporary – resets every session | Persistent – retained across sessions |
| **Scope** | Flat and linear – treats all tokens equally, no sense of priority | Hierarchical and structured – prioritizes important details |
| **Scaling Cost** | High – increases with input size | Low – only stores relevant information |
| **Latency** | Slower – larger prompts add delay | Faster – optimized and consistent |
| **Recall** | Proximity based – forgets what's far behind | Intent or relevance based |
| **Behavior** | Reactive – lacks continuity | Adaptive – evolves with every interaction |
| **Personalization** | None – every session is stateless | Deep – remembers preferences and history |

# Why RAG is Not the Same as Memory

Both RAG (Retrieval-Augmented Generation) and memory systems retrieve information to support an LLMs, they solve very different problems.

- RAG brings external knowledge into the prompt at inference time. It's useful for grounding responses with facts from documents.
- But RAG is fundamentally stateless - it has no awareness of previous interactions, user identity, or how the current query relates to past conversations.
- Memory brings in continuity. It captures user *preferences, past queries, decisions, and failures* and makes them available in future interactions.

**RAG helps the agent answer better. Memory helps the agent behave smarter.**

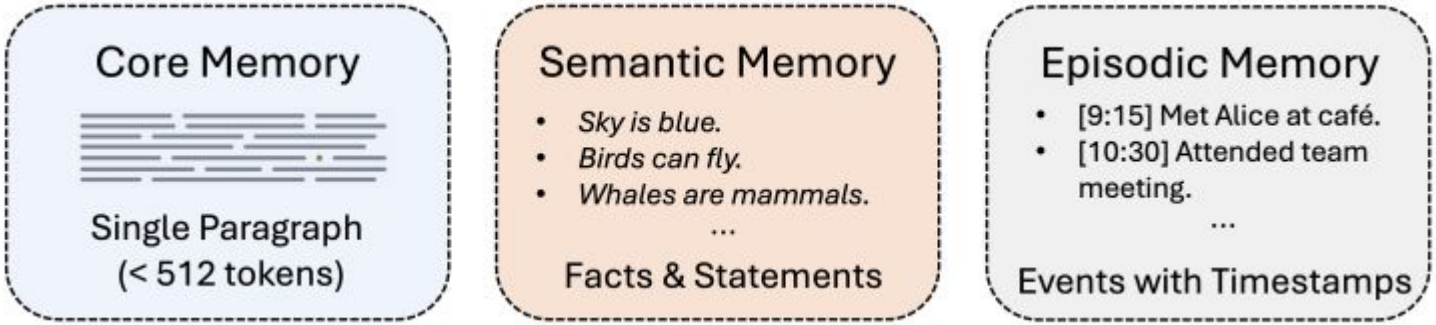
# Why RAG is Not the Same as Memory

| Aspect | RAG: Retrieval-Augmented Generation | Memory in Agents |
|---|---|---|
| **Temporal Awareness** | No concept of time or sequence | Tracks order, timing, and evolution of interactions |
| **Statefulness** | Stateless; each query is independent | Stateful; context accumulates across sessions |
| **User Modeling** | Task-bound; agnostic to user identity | Learns and evolves with the user |
| **Adaptability** | Cannot learn from past interactions | Adapts based on what worked or failed |

# Types of Memory in Agents: A High-Level Taxonomy

At a foundational level, memory in AI agents comes in two forms:

- Short-term memory: Holds immediate context within a single interaction.
- Long-term memory: Persists knowledge across sessions, tasks, and time:




Just like in humans, these memory types serve different cognitive functions. Short-term memory helps the agent stay coherent in the moment. Long-term memory helps it learn, personalize, and adapt.

# Types of Memory in Agents: A High-Level Taxonomy

| Type | Role | Example |
| --- | --- | --- |
| **Working Memory** (short-term) | Maintains short-term conversational coherence | “What was the last question again?” |
| **Factual Memory** (long-term) | Retains user preferences, communication style, domain context | “You prefer markdown output and short-form answers.” |
| **Episodic Memory** (long-term) | Remembers specific past interactions or outcomes | “Last time we deployed this model, the latency increased.” |
| **Semantic Memory** (long-term) | Stores generalized, abstract knowledge acquired over time | “Tasks involving JSON parsing usually stress you out, want a quick template?.” |

# Summary: Memory Is the Foundation, Not a Feature

In a world where every agent has access to the same models and tools, memory will be the differentiator. Not just the agent that responds — the one that remembers, learns, and grows with you will win.

It’s not a feature or bonus capability for elite agents. It’s the foundation that transforms agents from disposable tools into enduring teammates.

# Part 2. How to Benchmark Agent Memory?

# Agent Memory - Benchmarks

Some work uses long context QA and compare with RAG baselines.
https://arxiv.org/pdf/2509.25911

| Method | Metric | AR | | | TTL | | | LRU | Avg. |
|---|---|---|---|---|---|---|---|---|---|
| | | SQuAD | HotpotQA | PerLTQA | TREC-C | NLU | Pubmed | BookSum | |
| Long-Context | Perf. | 0.742 | **0.852** | 0.605 | 0.623 | **0.708** | 0.533 | 0.052 | 0.588 |
| | Mem. | 10.6K | 9.7K | 13.1K | 3.9K | 6.1K | 16.7K | 15.4K | 10.8K |
| RAG-Top2 | Perf. | 0.762 | 0.849 | 0.623 | 0.612 | 0.508 | **0.570** | 0.042 | 0.567 |
| | Mem. | 10.6K | 9.7K | 16.7K | 3.9K | 6.1K | 16.7K | 15.6K | 11.3K |
| MemAgent | Perf. | 0.091 | 0.140 | 0.052 | 0.562 | 0.290 | 0.343 | 0.103 | 0.236 |
| | Mem. | 0.79K | 0.76K | 0.29K | 1.24K | 0.99K | 0.94K | 0.59K | 0.84K |
| MEM1 | Perf. | 0.039 | 0.083 | 0.068 | 0.269 | 0.056 | 0.175 | 0.085 | 0.111 |
| | Mem. | 0.16K | 0.22K | 0.14K | 0.23K | 0.22K | 0.08K | 0.16K | 0.17K |
| Mem-$\alpha$ | Perf. | **0.786** | 0.832 | **0.659** | **0.666** | 0.658 | 0.545 | **0.187** | **0.642** |
| | Mem. | 10.1k | 8.7k | 11.2k | 4.0k | 6.5k | 12.3k | 2.2k | 7.9k |

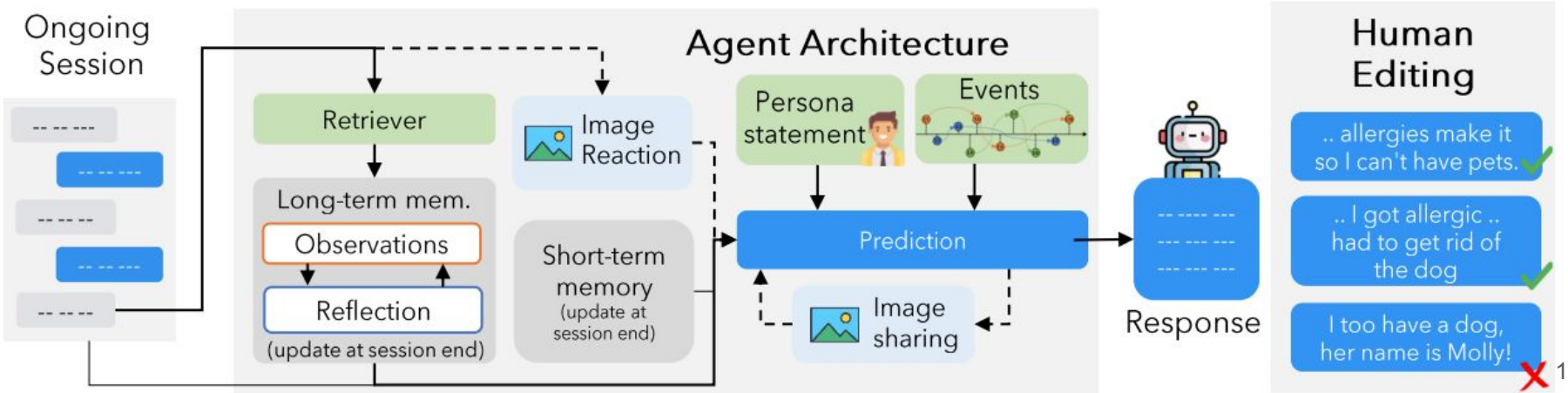
# Agent Memory Benchmark - LoCoMo

LoCoMo has 10 conversations, where each conversation has 600 dialogue turns and 26000 tokens on average. There are averagely 200 questions for each conversation.

Four types of questions are included: Temporal, Single Hop, Multi-Hop, and Open-domain.

# Agent Memory Benchmark - LoCoMo Construction

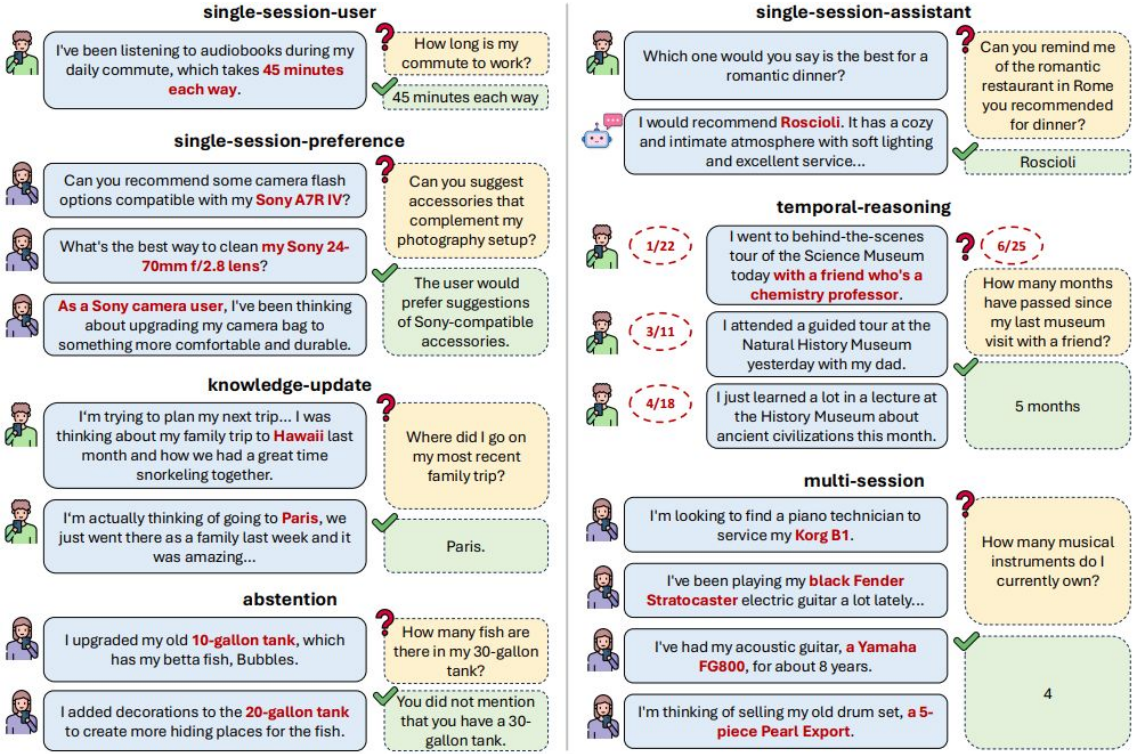
- assigned a **distinct persona** and a timeline of connected **events** in their file.
- equipped with a **memory and reflection module** for dialog generation
- enabled for image-sharing and image-reaction behaviors (left).
- The generated conversations are edited by human annotators to maintain long-range consistency (right).



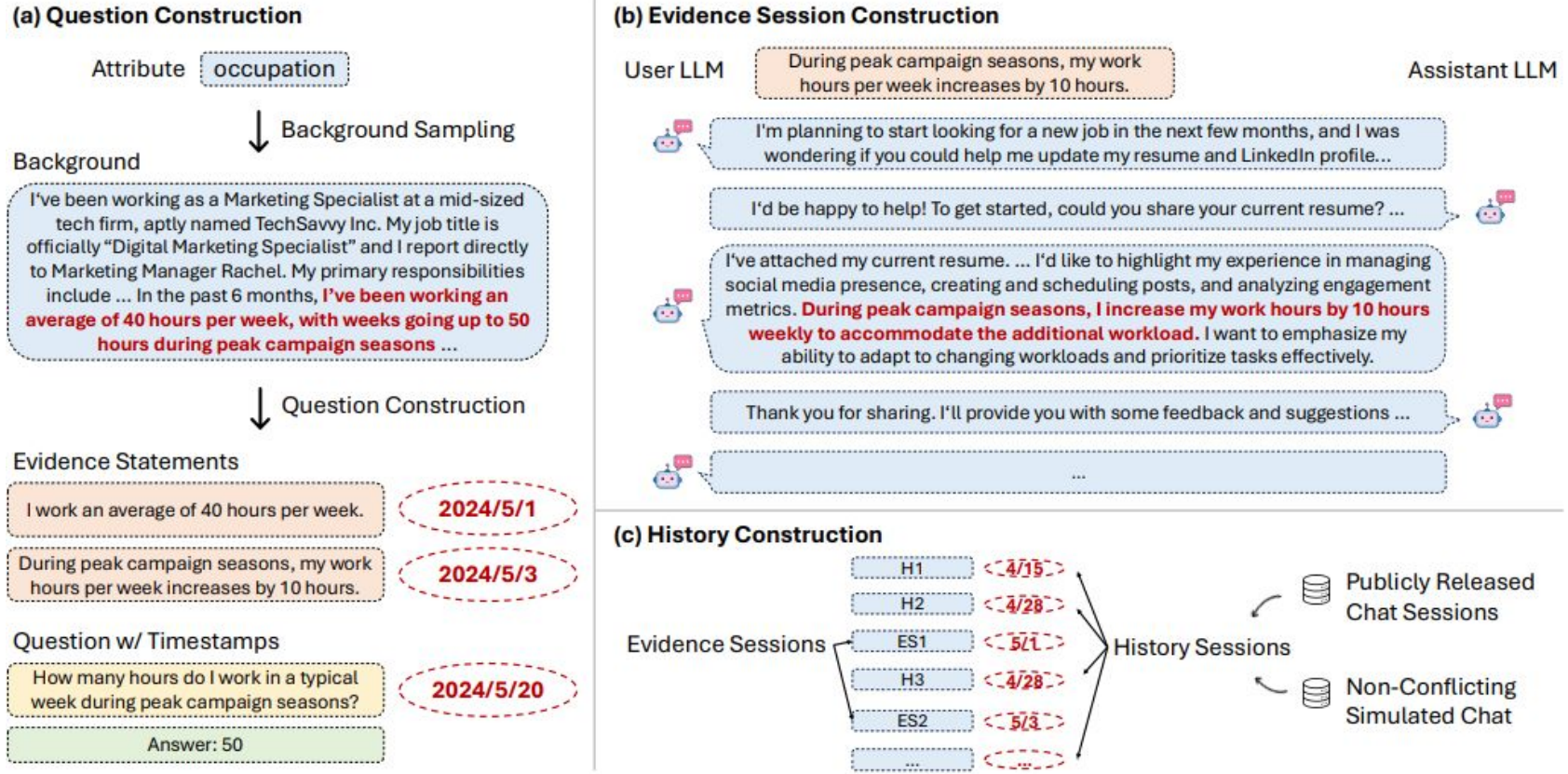
# Agent Memory Benchmark - LongMemEval

LongMemEval has 50k sessions, and 500 questions for them. The questions are human annotated, and sessions are LLM-generated

Information Extraction (IE), Multi-Session Reasoning (MR): Knowledge Updates (KU): Temporal Reasoning (TR)Abstention (ABS)



# Agent Memory Benchmark - LongMemEval Construction



# Q: The problem with the existing benchmarks?

# Agent Memory Benchmark - MemoryArena

**Environments and Tasks**

### Env. 1: Bundled Web Shopping

**Task:** Buy me a camera bundle

**Sub-Task 1:**

*"Buy a Camera Body; Budget=800usd; Buy the cheapest one; my options are:*

*A. A Nikon D5600 DSLR camera...; B. A Canon EOS Rebel T7 DSLR camera...; C. ..."*

[Constraint: cheapest price]

Log:
[search] ... [click]...
[search] .... [click]...
[search] ... ... ...
[buy]...

*Agent: "bought Canon EOS Rebel ...".*

**Subtask. 2:**

*"Buy me* Camera Lens; *Buy the highest-rated one; my options are:*
*A Canon EF 24mm f/1.4L II USM ...; B. A Sigma 35mm F1.4 Art DG HSM lens for Canon ...; C. A AF-P DX 70-300mm f/4.5-6.3G ED for Nikon... ; D.....*

[Constraint: highest rating; compatible with the camera body (Canon EOS Rebel...) ]

Log:
[search] ... [click]...
[search] ....
[click]... ... ... ...
[buy]...

***Subtask 3: Buy me ...***

...

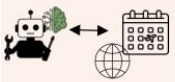
### Env. 2: Group Travel Planning

**Task:** Plan a group travel with shared/distinct preferences

**Sub-Task 1:**

"I am Tracy. A 5-day travel itinerary; Single-person trip; from Orlando; touring 2 cities in Texas; Date 03/10/25-03/04/25; Budget: 3,100.

[Constraint: trip length, destination, ... ]



Log:
[search flight] ...
[search hotel]...
[search Restaurant]
[search ...

*Agent: Tracy's itinerary ...*

**Subtask. 2:**

*"I am Chelsea. I'm joining Tracy for this trip. I like luxury hotels. So I can spend $150 more than Tracy's first-day accommodation. I have plan for the second day. So I will try Chelsea's second-day lunch restaurant in my third day; I want to try Korean BBQ with rating 4.5 and above in my third day."*

[Constraints: (join shared activities); (Plan individual activities based on preferences]

Log:
[search flight] ...
[search hotel]...
[search Restaurant]
[search ...

***Subtask 3: I'm Emily ....***

...

### Env. 3: Progressive Web Search

**Task:** Find a person name satisfying all conditions

**Sub-Task 1:**

"What is the name of the person who completed their PhD in 1989?"

[Constraints: PhD, 1989]

Agent: [Name list]

Log:
[search] ...
[reasoning]...
[search] ...

**Sub-Task 2:**

"Among them, what is the name of the person who published a book in 2014?"

[Constraints: publish book, 2014]

Log:
[search] ...
[reasoning]...
[search] ...

Agent: [Name list]

**Sub-Task 3:**

Among them what is the name of the person who was a keynote speaker at a conference in 2012?

[Constraints: keynote speaker, conf. 2002]

Log:
[search] ... ...

***Subtask 4: What is the name of ....***

...

### Env. 4 (Math): Formal Reasoning

**Task:** Express the upper bound of the run-time v.s. sample size tradeoffs implied for linear-memory algorithms for Tensor PCA

**Sub-Task 1:**

"[necessary definitions]
for In Hermite Decomposition for Tensor PCA, express $\frac{d\mu_v}{d\mu_0}(X) =?$ For any $X \in \otimes^k \mathbb{R}^d$
"

Log:
[reasoning] ...
[calculation] ...

Agent: [reasoning Trace]

$\frac{d\mu_v}{d\mu_0}(X) = \sum_{i=0}^{\infty} \frac{\lambda^i}{\sqrt{i!}} \cdot H_i\left(\frac{\langle X, V^{\otimes k}\rangle}{\sqrt{d^k}}\right)$

For any $X \in \otimes^k \mathbb{R}^d$

**Sub-Task 2:**

"[necessary definitions]
Use the previous derivation to calculate the following terms in terms of the integrated Hermite polynomials: $\mathrm{E}_0\left[\overline{H_i}(X;S) \cdot \overline{H_j}(X;S)\right]$
"

Log:
[reasoning] ...
[calculation] ...

Agent: [reasoning Trace]

$\mathrm{E}_0\left[\overline{H_i}(X;S) \cdot \overline{H_j}(X;S)\right] = 0$

***Subtask 3: ...***

...

### Env. 4 (Phys.): Formal Reasoning

**Task:** Find the independent null constraints on the spectral density for Higgs scattering

**Sub-Task 1:**

"[necessary definitions]
For singlet scalar scattering, there exists a set of non-trivial constraints on $\rho_\ell(\mu)$ from the sum rule and crossing symmetry. What are the independent null constraints?
"

Log:
[reasoning] ...
[calculation] ...

Agent: [reasoning Trace]

$B_{ijkl}(s,t) = B_{ikjl}(t,s)$

**Sub-Task 2:**

"For scalar doublets, what are the independent null constraints on $\rho_\ell^{1122}(\mu)$, $\rho_\ell^{1212}(\mu)$, and $\rho_\ell^{1221}(\mu)$?"

Log:
[reasoning] ...
[calculation] ...

Agent: [reasoning Trace]

$c_{ijkl}^{m,n} = \left[\rho_\ell^{ijkl}(\mu) + (-1)^m \rho_\ell^{ikjl}(\mu)\right] \sum_{p=0}^{n} \frac{L_\ell^p H_{m+1}^{n-p}}{\mu^{m+n+1}}$

***Subtask 3: ...***

...

# Agent Memory Benchmark - MemoryArena

SOTA model only achieves 0.02 Success Rate

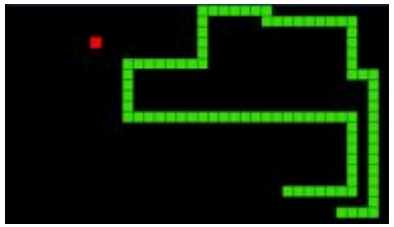
| | Memory Type | Bundled web shopping | | Group Travel Planing | | | Progressive Web Search | | Formal Reasoning | | | | All Task Avg SR |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | | | | Math | | Phys | | |
| | | SR | PS | SR | PS | sPS | SR | PS | SR | PS | SR | PS | |
| **Task agent + Long Context** | | | | | | | | | | | | | |
| **GPT-5.1-mini** | 0D | 0.01 | 0.58 | 0.00 | 0.00 | 0.52 | 0.06 | 0.05 | 0.26 | 0.38 | 0.45 | 0.6 | 0.16 |
| **GPT-4.1-mini** | 0D | 0.00 | 0.43 | 0.00 | 0.00 | 0.19 | 0.02 | 0.03 | 0.19 | 0.34 | 0.4 | 0.55 | 0.12 |
| **Gemini-3-Flash** | 0D | **0.12** | 0.76 | 0.00 | 0.01 | **0.62** | 0.07 | 0.04 | 0.16 | 0.30 | 0.5 | 0.55 | 0.17 |
| **Claude-Sonnet-4.5** | 0D | **0.12** | **0.79** | 0.00 | **0.06** | 0.44 | 0.02 | 0.03 | 0.29 | 0.31 | 0.50 | 0.60 | 0.19 |
| **Long Context Avg** | | 0.06 | 0.64 | 0.00 | 0.02 | 0.44 | 0.04 | 0.04 | 0.23 | 0.33 | 0.46 | 0.58 | |
| **Task Agent + Memory Agents** | | | | | | | | | | | | | |
| **Letta** | 1D | 0.00 | 0.5 | 0.00 | 0.00 | 0.35 | 0.16 | 0.09 | 0.13 | 0.31 | 0.45 | 0.65 | 0.15 |
| **Mem0** | 1D | 0.00 | 0.45 | 0.00 | 0.00 | 0.24 | 0.24 | 0.09 | 0.19 | 0.34 | 0.25 | 0.43 | 0.14 |
| **Mem0-g** | 2D | 0.00 | 0.43 | 0.00 | 0.00 | 0.30 | 0.15 | 0.08 | 0.19 | 0.32 | 0.25 | 0.50 | 0.12 |
| **Reasoning Bank** | 1D | 0.00 | 0.27 | 0.00 | 0.00 | 0.00 | 0.10 | 0.06 | 0.23 | **0.35** | 0.25 | 0.45 | 0.12 |
| **Memory Avg** | | 0.00 | 0.41 | 0.00 | 0.00 | 0.25 | 0.15 | 0.08 | 0.18 | 0.33 | 0.30 | 0.51 | |
| **Task Agent + RAG Systems** | | | | | | | | | | | | | |
| **BM25** | 0D | 0.00 | 0.56 | 0.00 | 0.01 | 0.45 | **0.28** | 0.09 | 0.23 | **0.39** | 0.45 | 0.58 | 0.19 |
| **Text-Embedding-3-Small** | 0D | 0.00 | 0.55 | 0.00 | 0.01 | 0.50 | 0.23 | 0.09 | **0.32** | 0.36 | **0.6** | **0.7** | **0.23** |
| **MemoRAG** | 1D | 0.00 | 0.54 | 0.00 | 0.03 | 0.50 | 0.22 | **0.21** | 0.23 | **0.39** | 0.50 | 0.67 | 0.19 |
| **GraphRAG** | 2D | 0.00 | 0.52 | 0.00 | 0.01 | 0.51 | 0.04 | 0.05 | 0.26 | **0.39** | 0.55 | 0.63 | 0.17 |
| **RAG Avg** | | 0.00 | 0.54 | 0.00 | 0.02 | 0.49 | 0.19 | 0.11 | 0.26 | 0.38 | 0.53 | 0.65 | |
| **All Method Avg** | | 0.02 | 0.52 | 0.00 | 0.02 | 0.38 | 0.23 | 0.09 | 0.22 | 0.35 | 0.42 | 0.57 | |

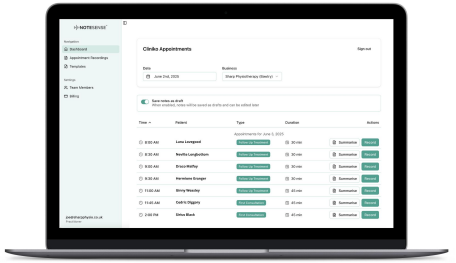
# Agent Memory Benchmark - VibeCodingMem (ours)

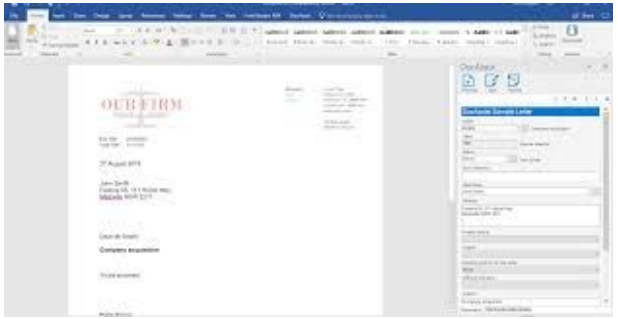
Vibe coding dialogue of SnakeGame, DocAssist, NoteSense, etc.

Using Claude CLI, with code source, files, and documents as additional input. An agentic environment.

Question types: Single hop, Multi-hop, Single session, Cross session, User Preference, Open-domain, Knowledge facts, Temporal.







# Agent Memory Benchmark - VibeCodingMem Construction

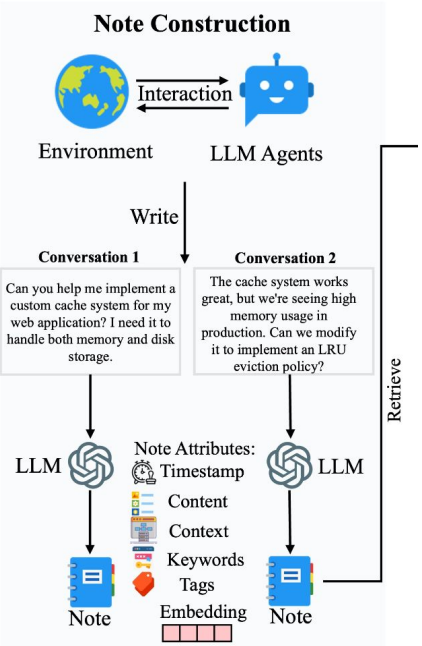
- Feature-based sessions
- Human annotated questions, evidence, and types


```
{
  "question": "What migration or backfill steps were needed across sessions?",
  "category": [
    "cross-session",
    "temporal",
    "multihop"
  ],
  "evidence": [
    "D3:6: \"Backfill existing notes with 'created' activity\"",
    "D4:1: \"Migrate planner.json tasks to task db\"",
    "D5:1: \"existing notes also need to be persisted into the database... there needs to be a migration\""
  ],
  "answer": "Backfill activity_history for existing notes; migrate planner_items to task DB; migrate notes to database."
},
```

# Part 3. How Are the Agent Memory Frameworks Designed?

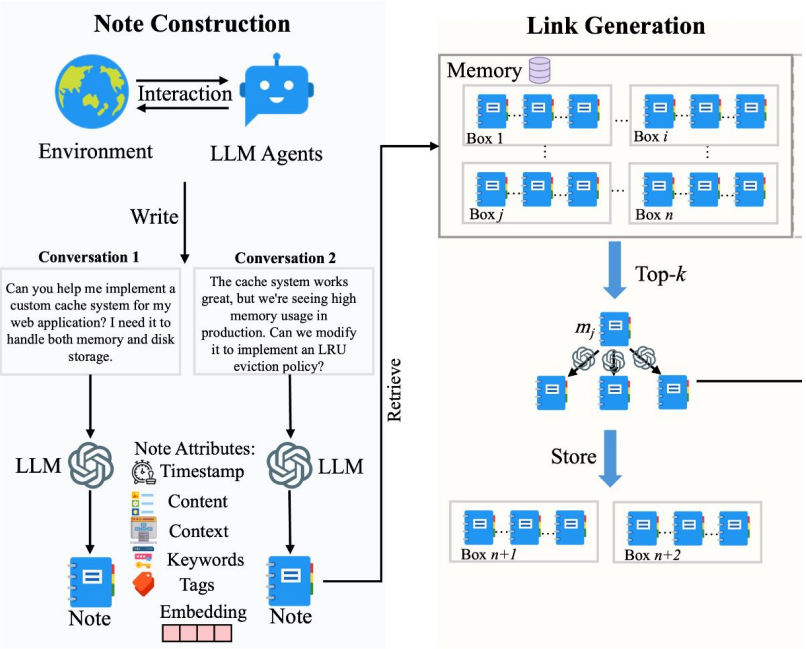
# Agent Memory Framework: A-Mem

Note Construction: use LLM to transform new conversation to a note（content、context、keywords、tags、timestamp）



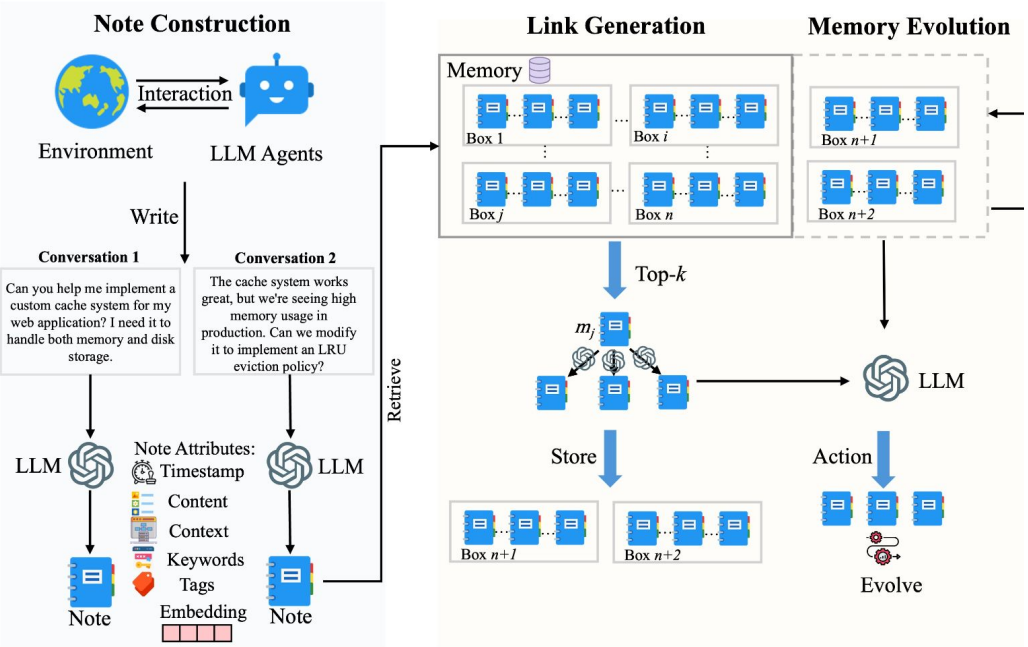
# Agent Memory Framework: A-Mem

Link Generation: Find relevant notes in the DB, build links between them



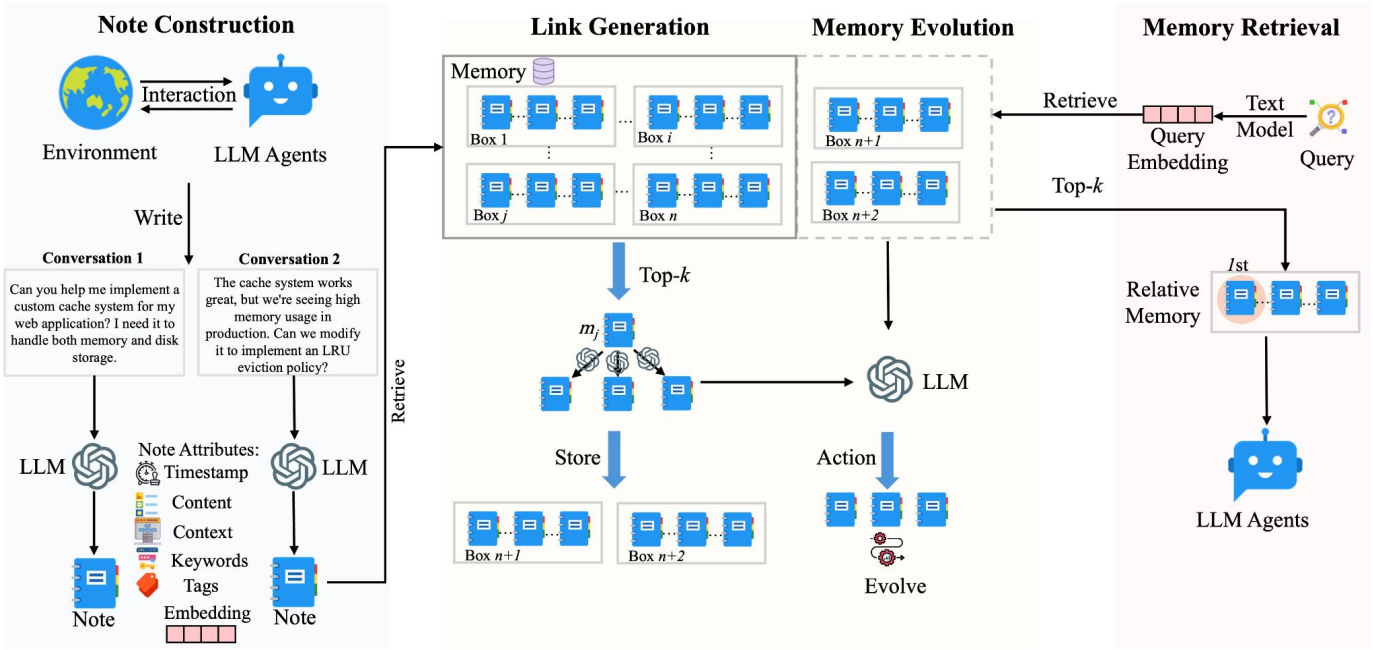
# Agent Memory Framework: A-Mem

Memory Evolution: LLM decide if the neighborhood needs to be updated, if so, update neibors



# Agent Memory Framework: A-Mem

Memory Retrieval: Given a query, retrieve the top-k memory in the DB and feed to agents.



# Agent Memory Framework: Mem0

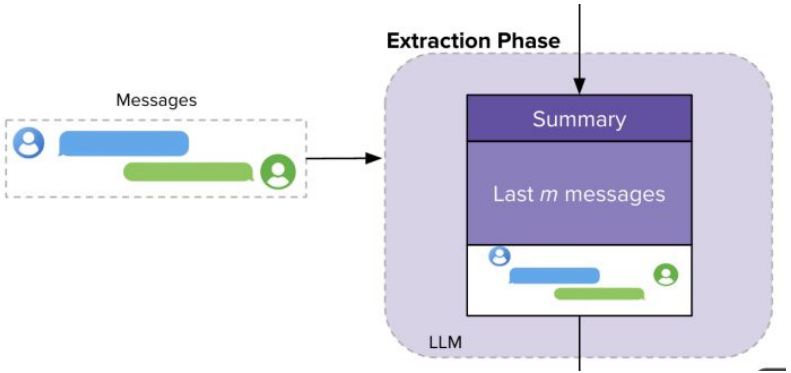
Achieve SOTA on three LLMs on LoCoMo dataset.

| Model | | Method | Category | | | | | | | | | | Average | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | Multi Hop | | Temporal | | Open Domain | | Single Hop | | Adversial | | Ranking | | Token |
| | | | F1 | BLEU | F1 | BLEU | F1 | BLEU | F1 | BLEU | F1 | BLEU | F1 | BLEU | Length |
| GPT | 4o-mini | LoCoMo | 25.02 | 19.75 | 18.41 | 14.77 | 12.04 | 11.16 | 40.36 | 29.05 | **69.23** | **68.75** | 2.4 | 2.4 | 16,910 |
| | | ReadAgent | 9.15 | 6.48 | 12.60 | 8.87 | 5.31 | 5.12 | 9.67 | 7.66 | 9.81 | 9.02 | 4.2 | 4.2 | 643 |
| | | MemoryBank | 5.00 | 4.77 | 9.68 | 6.99 | 5.56 | 5.94 | 6.61 | 5.16 | 7.36 | 6.48 | 4.8 | 4.8 | 432 |
| | | MemGPT | 26.65 | 17.72 | 25.52 | 19.44 | 9.15 | 7.44 | 41.04 | 34.34 | 43.29 | 42.73 | 2.4 | 2.4 | 16,977 |
| | | **A-Mem** | **27.02** | **20.09** | **45.85** | **36.67** | **12.14** | **12.00** | **44.65** | **37.06** | 50.03 | 49.47 | **1.2** | **1.2** | 2,520 |
| | 4o | LoCoMo | 28.00 | 18.47 | 9.09 | 5.78 | 16.47 | 14.80 | **61.56** | **54.19** | **52.61** | **51.13** | 2.0 | 2.0 | 16,910 |
| | | ReadAgent | 14.61 | 9.95 | 4.16 | 3.19 | 8.84 | 8.37 | 12.46 | 10.29 | 6.81 | 6.13 | 4.0 | 4.0 | 805 |
| | | MemoryBank | 6.49 | 4.69 | 2.47 | 2.43 | 6.43 | 5.30 | 8.28 | 7.10 | 4.42 | 3.67 | 5.0 | 5.0 | 569 |
| | | MemGPT | 30.36 | 22.83 | 17.29 | 13.18 | 12.24 | 11.87 | 60.16 | 53.35 | 34.96 | 34.25 | 2.4 | 2.4 | 16,987 |
| | | **A-Mem** | **32.86** | **23.76** | **39.41** | **31.23** | **17.10** | **15.84** | 48.43 | 42.97 | 36.35 | 35.53 | **1.6** | **1.6** | 1,216 |
| Qwen2.5 | 1.5b | LoCoMo | 9.05 | 6.55 | 4.25 | 4.04 | 9.91 | 8.50 | 11.15 | 8.67 | 40.38 | 40.23 | 3.4 | 3.4 | 16,910 |
| | | ReadAgent | 6.61 | 4.93 | 2.55 | 2.51 | 5.31 | 12.24 | 10.13 | 7.54 | 5.42 | 27.32 | 4.6 | 4.6 | 752 |
| | | MemoryBank | 11.14 | 8.25 | 4.46 | 2.87 | 8.05 | 6.21 | 13.42 | 11.01 | 36.76 | 34.00 | 2.6 | 2.6 | 284 |
| | | MemGPT | 10.44 | 7.61 | 4.21 | 3.89 | 13.42 | 11.64 | 9.56 | 7.34 | 31.51 | 28.90 | 3.4 | 3.4 | 16,953 |
| | | **A-Mem** | **18.23** | **11.94** | **24.32** | **19.74** | **16.48** | **14.31** | **23.63** | **19.23** | **46.00** | **43.26** | **1.0** | **1.0** | 1,300 |
| | 3b | LoCoMo | 4.61 | 4.29 | 3.11 | 2.71 | 4.55 | 5.97 | 7.03 | 5.69 | 16.95 | 14.81 | 3.2 | 3.2 | 16,910 |
| | | ReadAgent | 2.47 | 1.78 | 3.01 | 3.01 | 5.57 | 5.22 | 3.25 | 2.51 | 15.78 | 14.01 | 4.2 | 4.2 | 776 |
| | | MemoryBank | 3.60 | 3.39 | 1.72 | 1.97 | 6.63 | 6.58 | 4.11 | 3.32 | 13.07 | 10.30 | 4.2 | 4.2 | 298 |
| | | MemGPT | 5.07 | 4.31 | 2.94 | 2.95 | 7.04 | 7.10 | 7.26 | 5.52 | 14.47 | 12.39 | 2.4 | 2.4 | 16,961 |
| | | **A-Mem** | **12.57** | **9.01** | **27.59** | **25.07** | **7.12** | **7.28** | **17.23** | **13.12** | **27.91** | **25.15** | **1.0** | **1.0** | 1,137 |
| Llama 3.2 | 1b | LoCoMo | 11.25 | 9.18 | 7.38 | 6.82 | 11.90 | 10.38 | 12.86 | 10.50 | 51.89 | 48.27 | 3.4 | 3.4 | 16,910 |
| | | ReadAgent | 5.96 | 5.12 | 1.93 | 2.30 | 12.46 | 11.17 | 7.75 | 6.03 | 44.64 | 40.15 | 4.6 | 4.6 | 665 |
| | | MemoryBank | 13.18 | 10.03 | 7.61 | 6.27 | 15.78 | 12.94 | 17.30 | 14.03 | 52.61 | 47.53 | 2.0 | 2.0 | 274 |
| | | MemGPT | 9.19 | 6.96 | 4.02 | 4.79 | 11.14 | 8.24 | 10.16 | 7.68 | 49.75 | 45.11 | 4.0 | 4.0 | 16,950 |
| | | **A-Mem** | **19.06** | **11.71** | **17.80** | **10.28** | **17.55** | **14.67** | **28.51** | **24.13** | **58.81** | **54.28** | **1.0** | **1.0** | 1,376 |
| | 3b | LoCoMo | 6.88 | 5.77 | 4.37 | 4.40 | 10.65 | 9.29 | 8.37 | 6.93 | 30.25 | 28.46 | 2.8 | 2.8 | 16,910 |
| | | ReadAgent | 2.47 | 1.78 | 3.01 | 3.01 | 5.57 | 5.22 | 3.25 | 2.51 | 15.78 | 14.01 | 4.2 | 4.2 | 461 |
| | | MemoryBank | 6.19 | 4.47 | 3.49 | 3.13 | 4.07 | 4.57 | 7.61 | 6.03 | 18.65 | 17.05 | 3.2 | 3.2 | 263 |
| | | MemGPT | 5.32 | 3.99 | 2.68 | 2.72 | 5.64 | 5.54 | 4.32 | 3.51 | 21.45 | 19.37 | 3.8 | 3.8 | 16,956 |
| | | **A-Mem** | **17.44** | **11.74** | **26.38** | **19.50** | **12.53** | **11.83** | **28.14** | **23.87** | **42.04** | **40.60** | **1.0** | **1.0** | 1,126 |

# Agent Memory Framework: Mem0

Memorize - Extraction Phase:

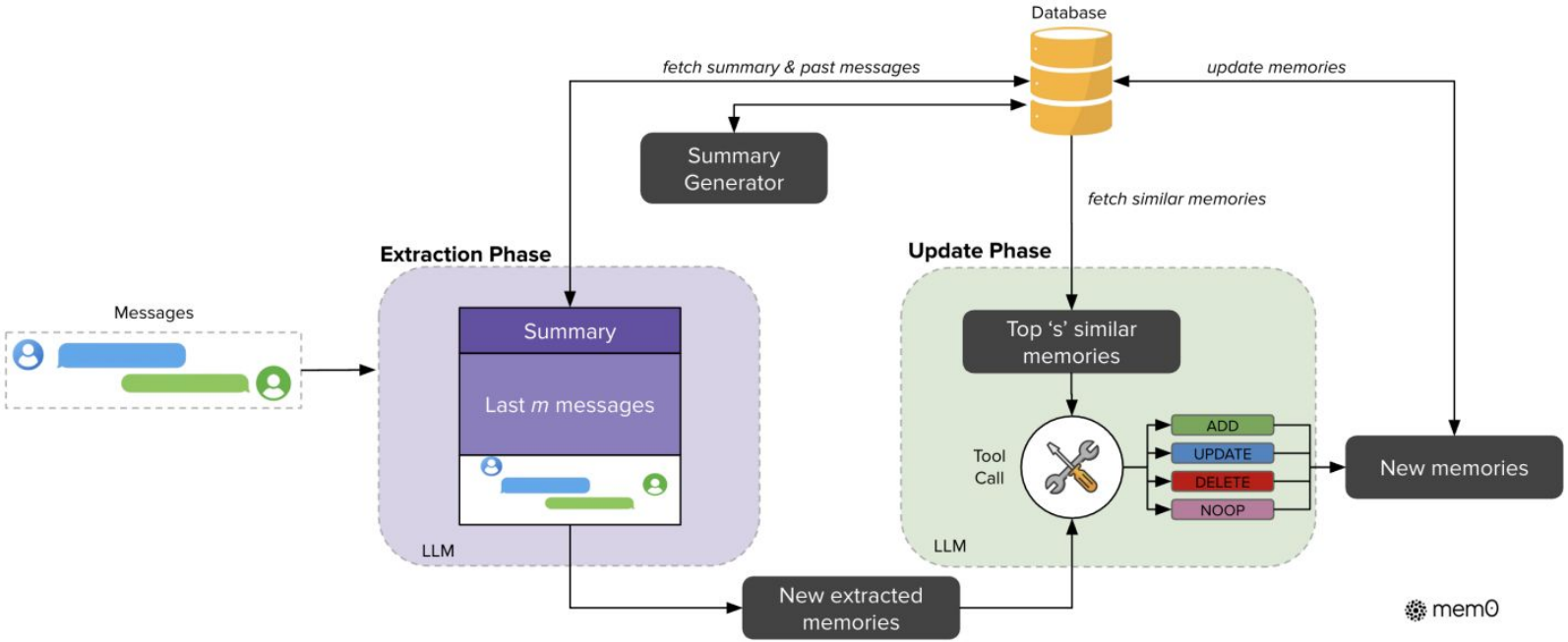
Session -> Process Conversation - >  LLM Call to extract Fact



# Agent Memory Framework: Mem0

Memorize - Update Phase:

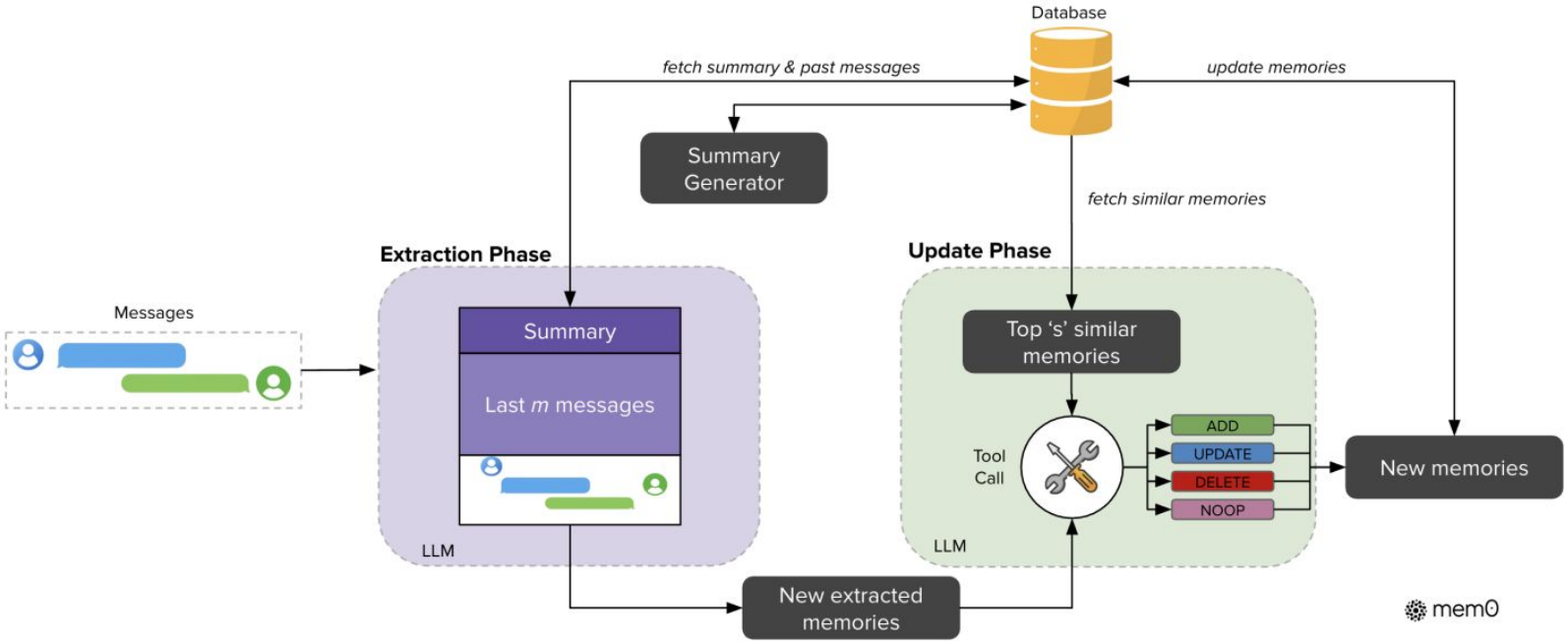
Find top 5 memory in the database -> Feed to LLM to decide tool call -> Save to DB



# Agent Memory Framework: Mem0

Search - Given a query, find the supporting memory

Search DB to get memory -> Feed to LLM to generate the answer



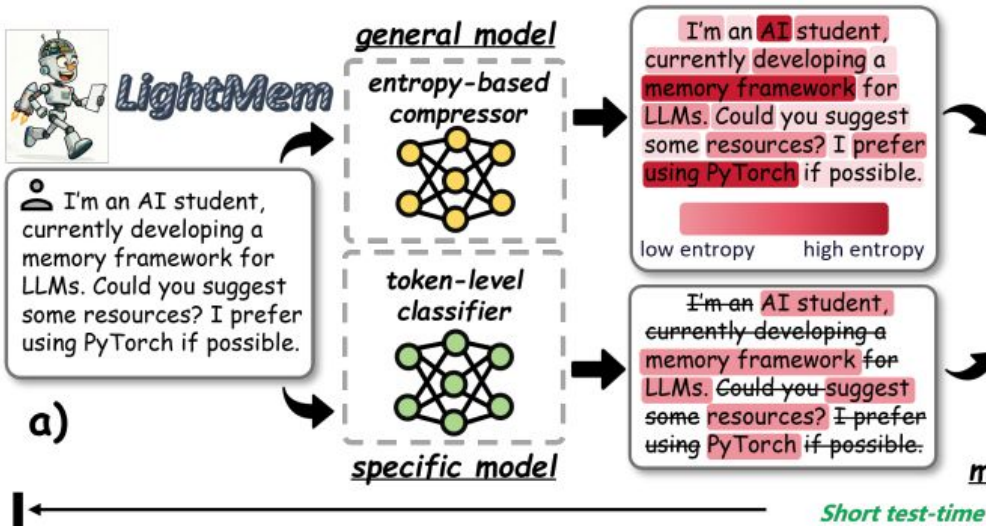
# Agent Memory Framework: Mem0

Mem0 further improves the A-Mem by a large Margin. It also provides a variant of using GraphDB, called Mem0^g.

| Method | Single Hop | | | Multi-Hop | | | Open Domain | | | Temporal | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | $F_1 \uparrow$ | $B_1 \uparrow$ | $J \uparrow$ | $F_1 \uparrow$ | $B_1 \uparrow$ | $J \uparrow$ | $F_1 \uparrow$ | $B_1 \uparrow$ | $J \uparrow$ | $F_1 \uparrow$ | $B_1 \uparrow$ | $J \uparrow$ |
| LoCoMo | 25.02 | 19.75 | – | 12.04 | 11.16 | – | 40.36 | 29.05 | – | 18.41 | 14.77 | – |
| ReadAgent | 9.15 | 6.48 | – | 5.31 | 5.12 | – | 9.67 | 7.66 | – | 12.60 | 8.87 | – |
| MemoryBank | 5.00 | 4.77 | – | 5.56 | 5.94 | – | 6.61 | 5.16 | – | 9.68 | 6.99 | – |
| MemGPT | 26.65 | 17.72 | – | 9.15 | 7.44 | – | 41.04 | 34.34 | – | 25.52 | 19.44 | – |
| A-Mem | 27.02 | 20.09 | – | 12.14 | 12.00 | – | 44.65 | 37.06 | – | 45.85 | 36.67 | – |
| A-Mem* | 20.76 | 14.90 | 39.79 ± 0.38 | 9.22 | 8.81 | 18.85 ± 0.31 | 33.34 | 27.58 | 54.05 ± 0.22 | 35.40 | 31.08 | 49.91 ± 0.31 |
| LangMem | 35.51 | 26.86 | 62.23 ± 0.75 | 26.04 | **22.32** | 47.92 ± 0.47 | 40.91 | 33.63 | 71.12 ± 0.20 | 30.75 | 25.84 | 23.43 ± 0.39 |
| Zep | 35.74 | 23.30 | 61.70 ± 0.32 | 19.37 | 14.82 | 41.35 ± 0.48 | **49.56** | 38.92 | **76.60 ± 0.13** | 42.00 | 34.53 | 49.31 ± 0.50 |
| OpenAI | 34.30 | 23.72 | 63.79 ± 0.46 | 20.09 | 15.42 | 42.92 ± 0.63 | 39.31 | 31.16 | 62.29 ± 0.12 | 14.04 | 11.25 | 21.71 ± 0.20 |
| Mem0 | **38.72** | **27.13** | **67.13 ± 0.65** | **28.64** | 21.58 | **51.15 ± 0.31** | 47.65 | 38.72 | 72.93 ± 0.11 | 48.93 | **40.51** | 55.51 ± 0.34 |
| Mem0$^g$ | 38.09 | 26.03 | 65.71 ± 0.45 | 24.32 | 18.82 | 47.19 ± 0.67 | 49.27 | **40.30** | 75.71 ± 0.21 | **51.55** | 40.28 | **58.13 ± 0.44** |

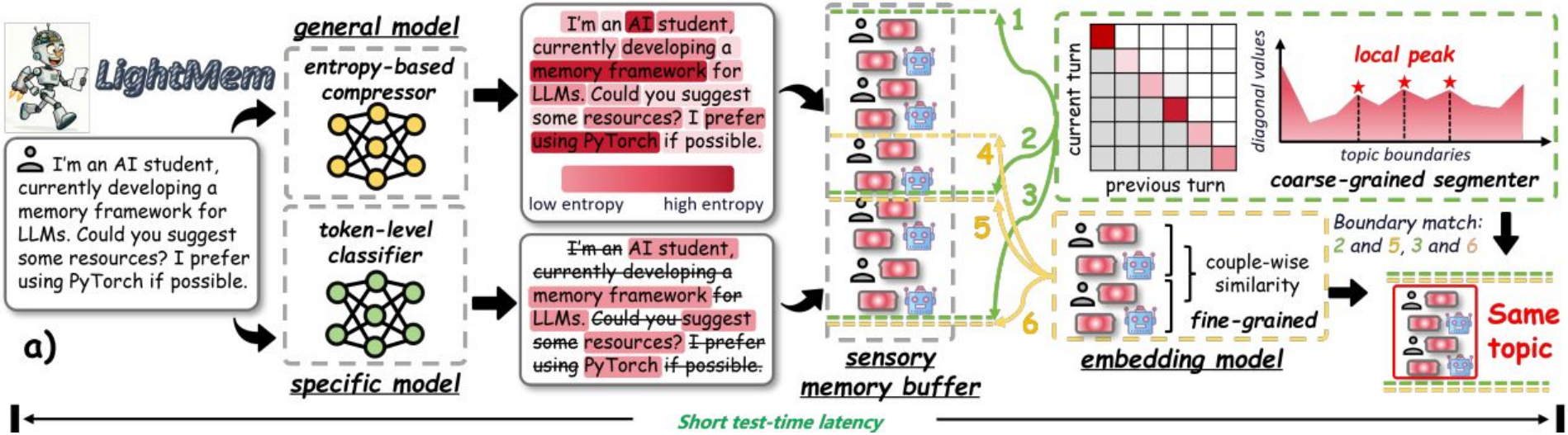
# Agent Memory Framework: LightMem

Compression of conversation: use two ways to compress the tokens



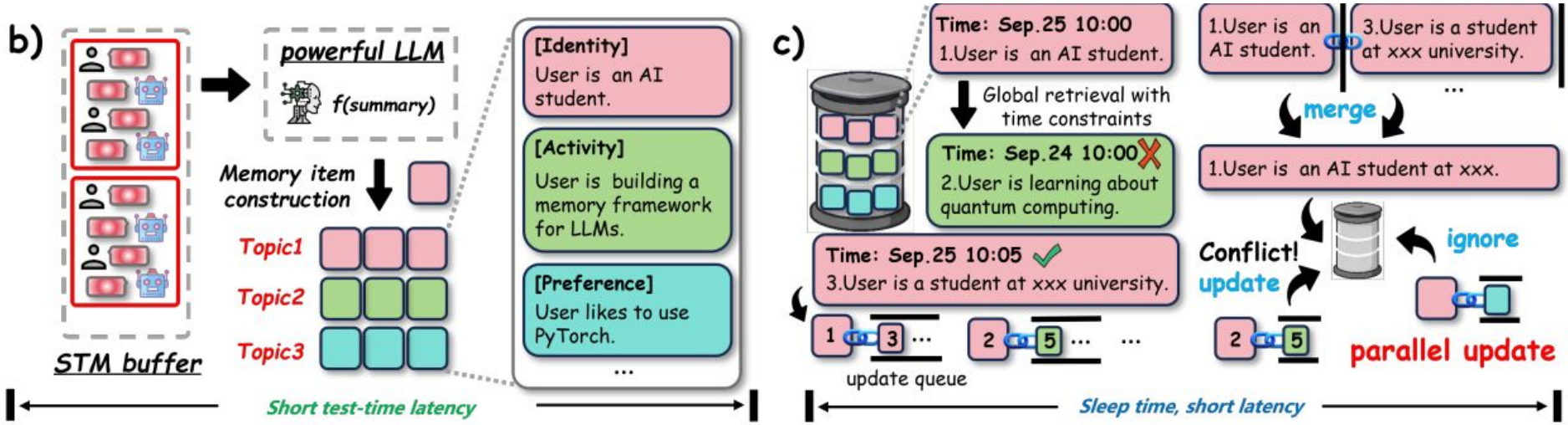
# Agent Memory Framework: LightMem

Compression of conversation -> Sensory Memory Buffer -> Topic Segmentation ->



# Agent Memory Framework: LightMem

Fact Extraction -> Store in Vector DB -> Offline Conflict Resolution and Update



# Agent Memory Framework: LightMem

r: Compression
th: STM buffer
OP-update: offline

Higher Performance
Lower Latency
Less API Calls

| Method | ACC (%) | Summary Tokens (k) | | Update Tokens (k) | | Total (k) | Calls | Runtime (s) |
|---|---|---|---|---|---|---|---|---|
| | | In | Out | In | Out | | | |
| GPT-4o-mini | | | | | | | | |
| FullText | 56.80 | – | – | – | – | 105.07 | – | – |
| NaiveRAG | 61.00 | – | – | – | – | – | – | 867.38 |
| LangMem | 37.20 | – | – | 982.68 | 119.48 | 1,102.16 | 520.62 | 2,293.70 |
| A-MEM | 62.60 | 214.66 | 42.82 | 1,157.52 | 190.81 | 1,605.81 | 986.55 | 5,132.06 |
| MemoryOS | 44.80 | 2,302.35 | 304.18 | 350.02 | 35.19 | 2,991.75 | 2,938.41 | 8,030.04 |
| Mem0 | 53.61 | 424.13 | 17.76 | 560.17 | 150.56 | 1,152.62 | 811.57 | 4,248.49 |
| LightMem | | | | | | | | |
| $r$=0.5, $th$=256 | 64.29 | 20.80 | 10.01 | – | – | 30.81 | 25.67 | 302.69 |
| (OP-update) | 64.69 | – | – | **44.46** | **2.56** | 47.02 | 70.23 | 342.63 |
| $r$=0.6, $th$=256 | 67.78 | 24.58 | 10.53 | – | – | 35.11 | 30.47 | 329.61 |
| (OP-update) | 65.39 | – | – | 53.98 | 3.18 | 57.16 | 85.07 | 411.56 |
| $r$=0.7, $th$=512 | **68.64** | **18.88** | **9.37** | – | – | **28.25** | **18.43** | **283.76** |
| (OP-update) | 67.07 | – | – | 79.38 | 4.06 | 83.44 | 125.47 | 496.03 |

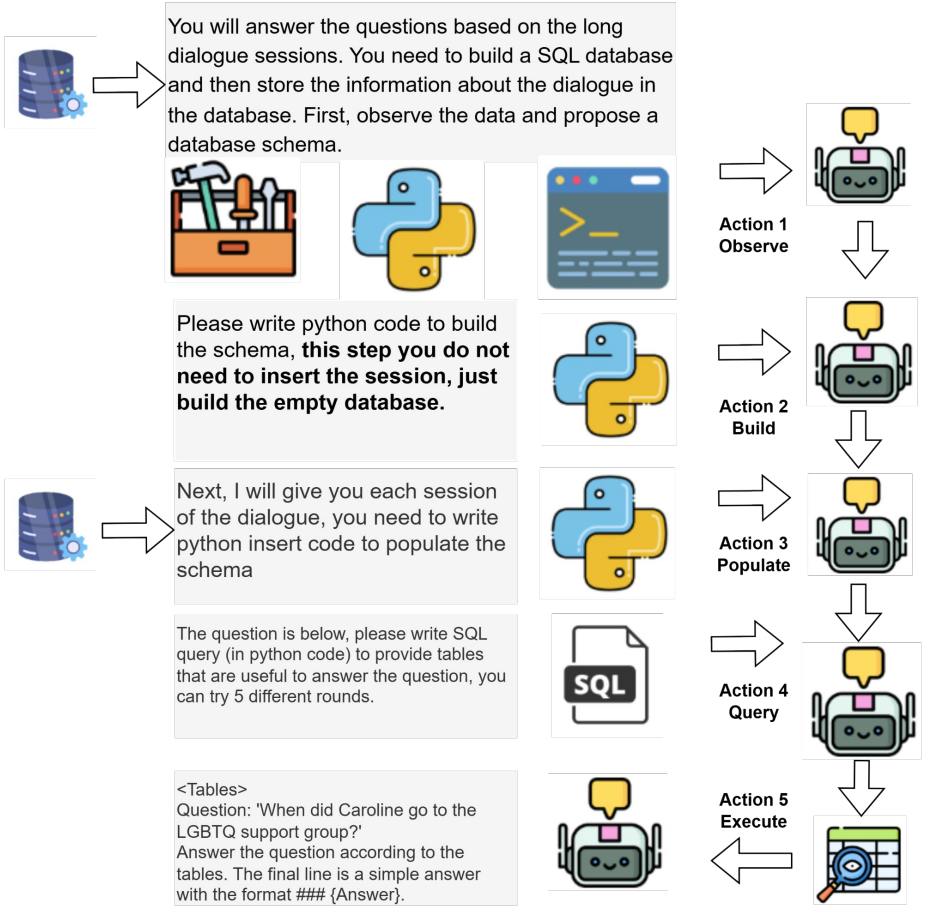
# Q: The problems with existing frameworks?

# Agentic Information Management

(ongoing work, please keep it confidential before publishing)

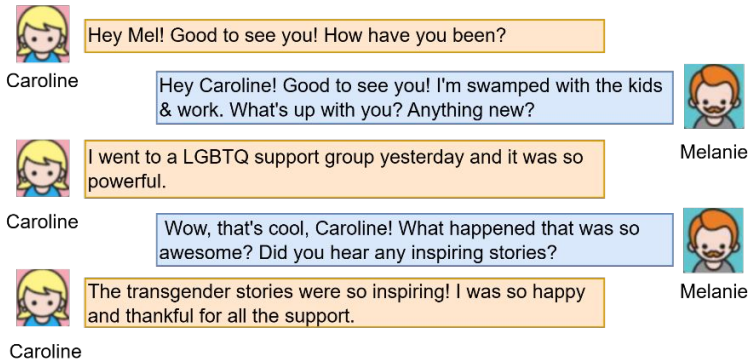
# Overview of AIM

- AIM proposes a pipeline that capture the structure most useful for supporting the target agent's tasks.
- Right figure shows the overview of the framework.
- Highly automated, Training-free, Adaptive to unseen tasks



# Stage 1: Observe Data and Build Data Model

Observe the source by reading a short preview.

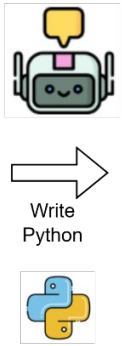


# Stage 1: Observe Data and Build Data Model

Observe the source by reading a short preview. Next, observe some samples queries.

# Stage 1: Observe Data and Build Data Model

Observe the source by reading a short preview. Next, observe some samples queries. The agent builds the schema given the observation..

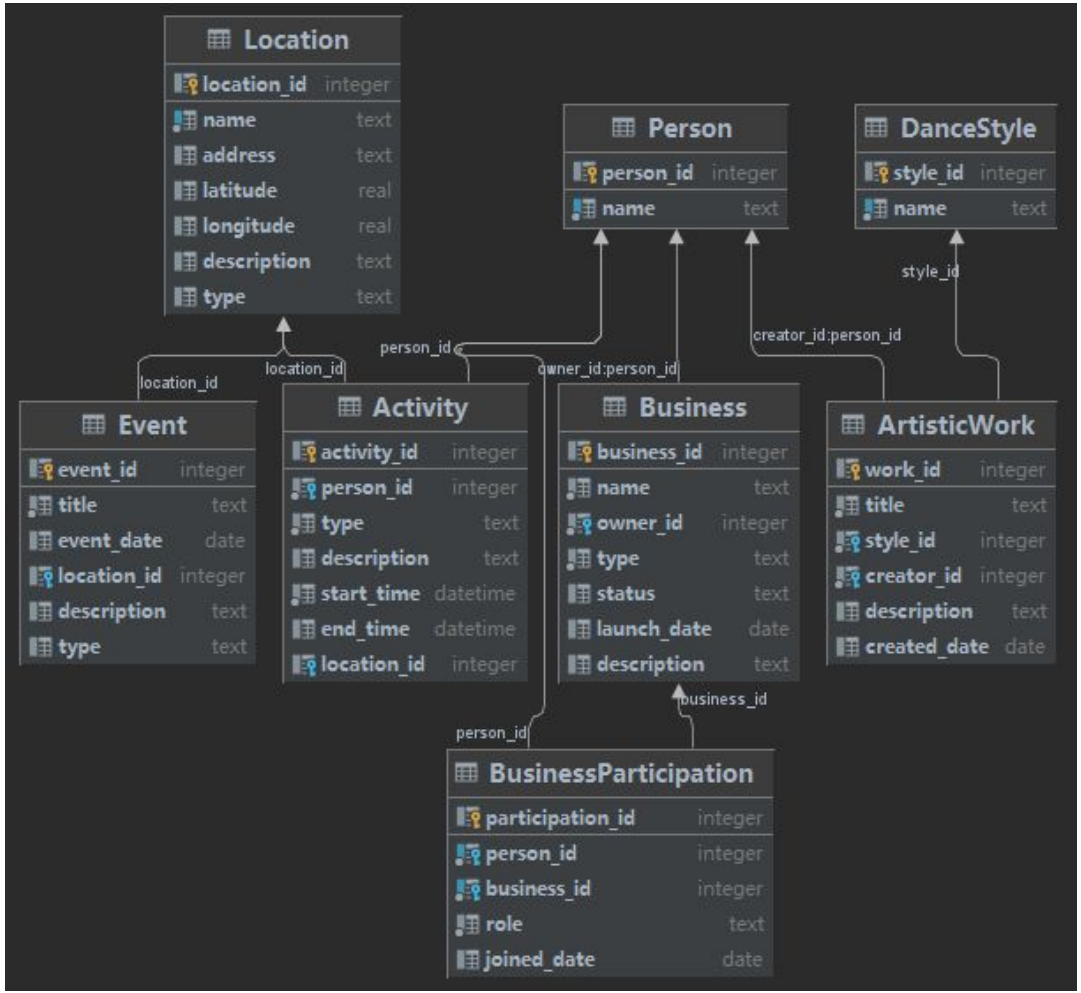

```python
import sqlite3

# Connect to the database (creates it if it doesn't exist)
conn = sqlite3.connect('dialogue_database.db')
cursor = conn.cursor()

# Create participants table
cursor.execute('''
CREATE TABLE IF NOT EXISTS participants (
    participant_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(100) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
''')
```

# Schema Example 1

7 tables

Foreign keys, Primary keys,

Capture different memories



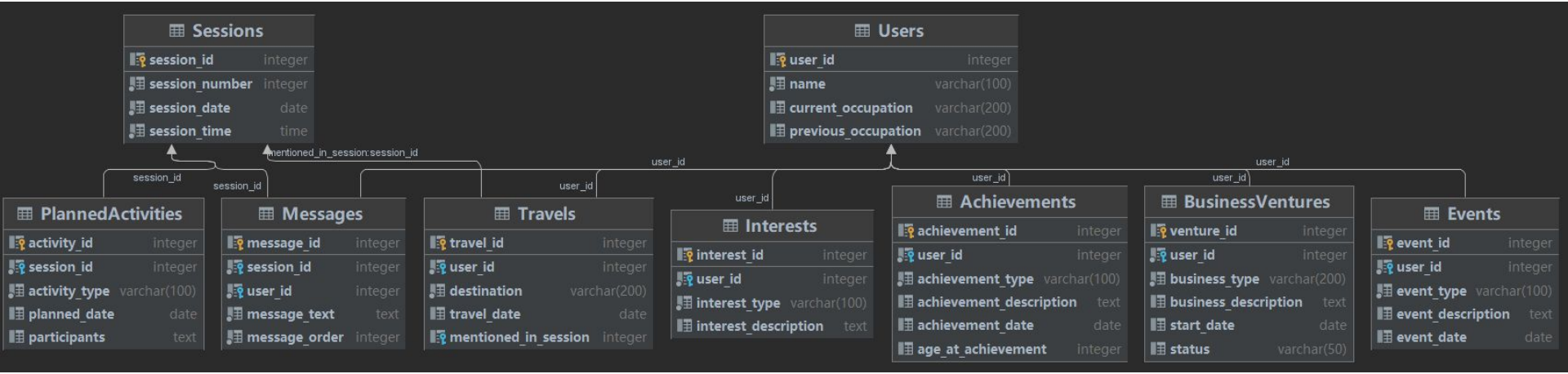
# Schema Example 2

9 tables

Foreign keys, Primary keys,

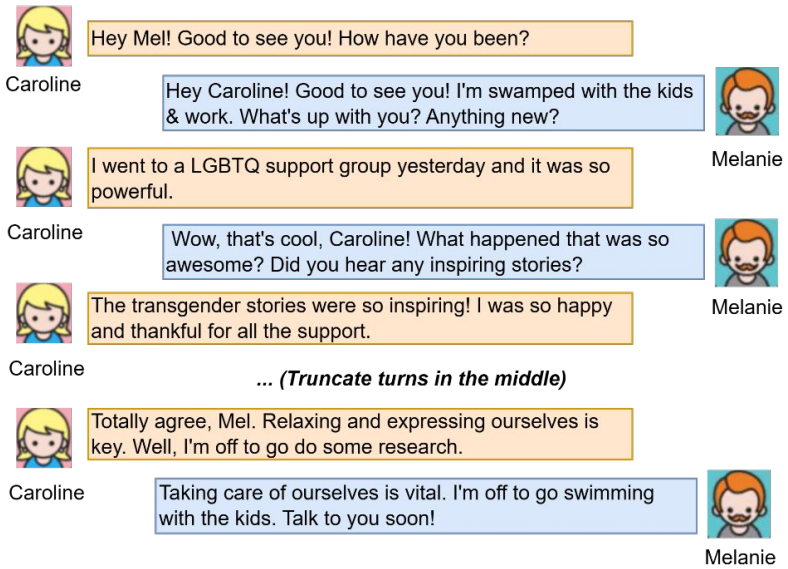
Capture different memories.

Generated tables are diverse



# Stage 2: Populate Sessions into Database

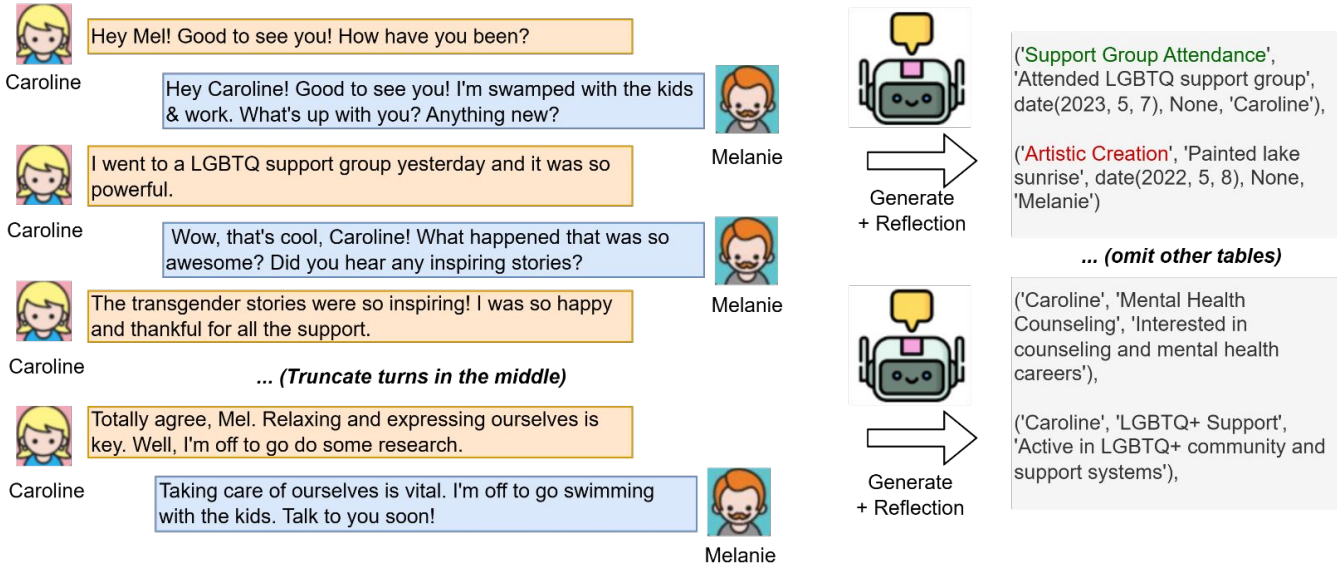
The agent reads the sessions one by one (a sample contains 20 sessions in LoCoMo [1] dataset, each with around 20 turns).




[1] Maharana, Adyasha, et al. "Evaluating very long-term conversational memory of llm agents." *ACL*. 2024.

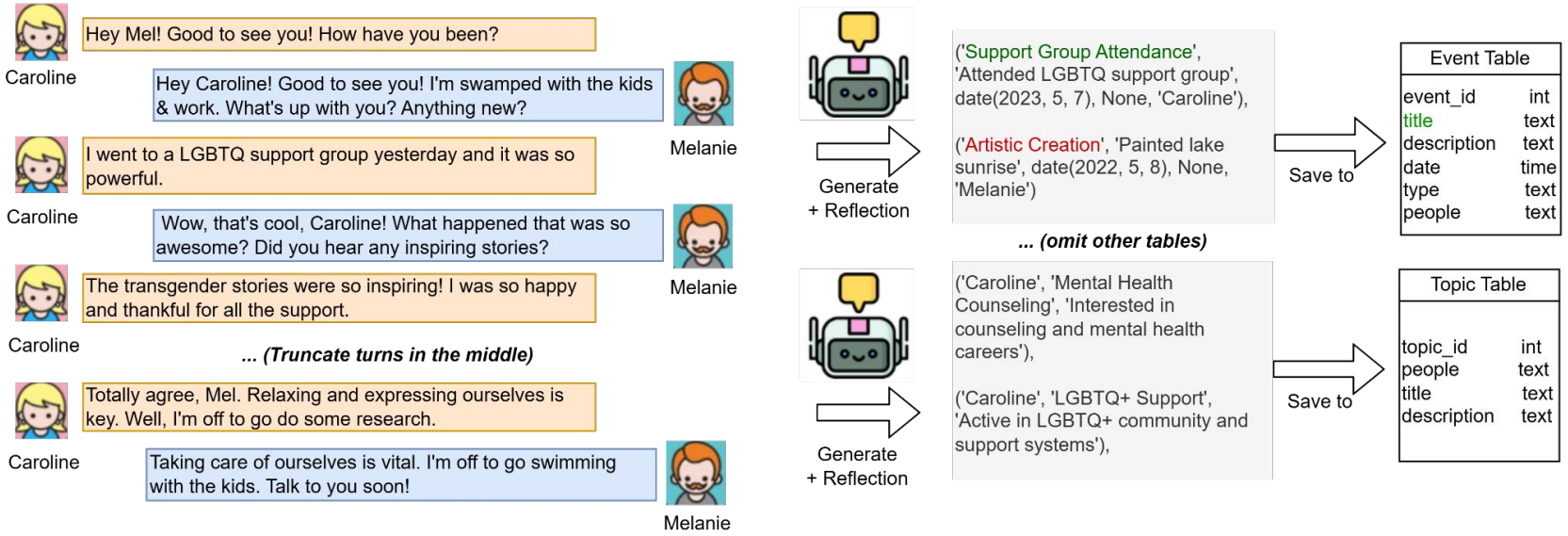
# Stage 2: Populate Sessions into Database

After reading each session, the agent extracts useful information from the session, and generate entries for each table. Agent will reflect multiple times for quality.



# Stage 2: Populate Sessions into Database

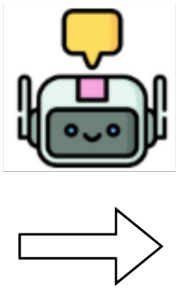
These entries will be saved to the different tables via SQL tools.



# Stage 3: Answer Query with the Database

In this stage, the executor will be given the query as well as the schema information built in previous session. Executor agent is allowed to explore the database for three times in order to answer the question.

When did Caroline go to the LGBTQ support group?

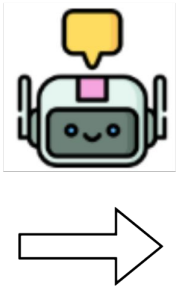
Iteration 1

```python
# Query 1: Check what participants exist
print("\n[Query 1] Checking participants in database:")
cursor.execute('SELECT * FROM participants LIMIT 5')
participants = cursor.fetchall()
print(f"Results found: {len(participants)}")
for row in participants:
    print(f"  {row}")
```

# Stage 3: Answer Query with the Database

In this stage, the executor will be given the query as well as the schema information built in previous session. Executor agent is allowed to explore the database for three times in order to answer the question.

When did Caroline go to the LGBTQ support group?

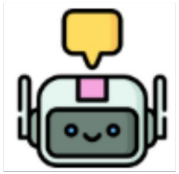
Iteration 2

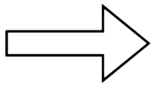
```python
# Query 2: Check what events exist
print("\n[Query 2] Checking events in database:")
cursor.execute('SELECT * FROM events LIMIT 5')
events = cursor.fetchall()
print(f"Results found: {len(events)}")
for row in events:
    print(f"  {row}")
```

# Stage 3: Answer Query with the Database

In this stage, the executor will be given the query as well as the schema information built in previous session. Executor agent is allowed to explore the database for three times in order to answer the question.

When did Caroline go to the LGBTQ support group?

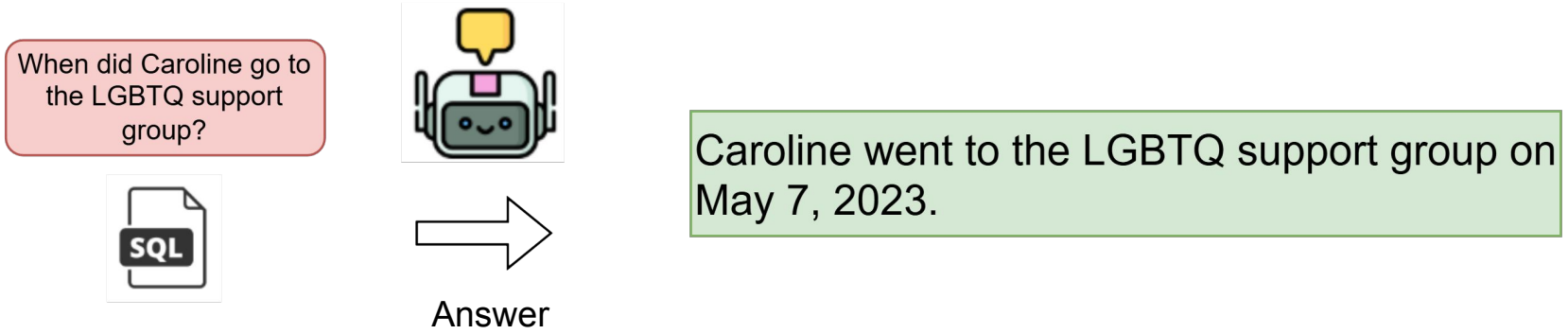


Iteration 3

```python
# Query 3: Search for Caroline and LGBTQ support group event
print("\n[Query 3] Searching for Caroline's LGBTQ support group event:")
cursor.execute('''
SELECT
    p.name,
    e.event_type,
    e.event_description,
    e.event_date,
    e.event_time
FROM events e
JOIN participants p ON e.participant_id = p.participant_id
WHERE p.name = 'Caroline'
AND e.event_type LIKE '%Support%'
LIMIT 5
''')
results = cursor.fetchall()
print(f"Results found: {len(results)}")
```

# Stage 3: Answer Query with the Database

Finally, the executor agent is asked to generate a short answer. The answer will be the output of the system.



# Experiments Settings

- Memory Baselines: Mem0 [1], A-Mem [2], GAM [3]
- RAG Baselines: RAG (with SOTA retriever Octen [4])
- Dataset: LoCoMo [5]
  - Long dialogue dataset for cross-session memory evaluation
  - Each sample contains 20 sessions (~500 dialogue turns)
  - QA dataset. Questions cover: **multi-hop reasoning, temporal reasoning** (with timestamps), **single-hop reasoning**, and **open-domain questions**.
- Evaluation: recall-focused human evaluation, focusing on whether the response covers the needed answer.


[1] Chhikara, Prateek, et al. "Mem0: Building production-ready ai agents with scalable long-term memory." *arXiv preprint arXiv:2504.19413* (2025).
[2] Xu, Wujiang, et al. "A-mem: Agentic memory for llm agents." *arXiv preprint arXiv:2502.12110* (2025).
[3] Yan, B. Y., et al. "General agentic memory via deep research." *arXiv preprint arXiv:2511.18423* (2025).
[4] Octen Series: Optimizing Embedding Models to #1 on RTEB Leaderboard
[5] Maharana, Adyasha, et al. "Evaluating very long-term conversational memory of llm agents." *ACL*. 2024.

# Experiments Results

- AIM outperforms all baselines by a large margin.
- The performance gain remains substantial even with weaker models (e.g., Qwen-3).
- For Haiku, performance is even better than Sonnet, likely because Haiku provides richer details for query generation.

| | multi | temp | open | single | average |
|---|---|---|---|---|---|
| **Claude-4.5-Sonnet** | | | | | |
| Mem0 | 33.44 | 25.95 | 44.62 | 50.00 | 38.50 |
| A-Mem | 42.19 | 59.46 | 84.62 | 65.00 | 62.82 |
| RAG | 71.25 | 67.57 | **88.46** | 90.00 | 79.32 |
| GAM | **76.56** | 85.14 | 84.62 | 80.00 | 81.58 |
| AIM | 71.56 | **94.59** | **88.46** | **95.00** | **87.40** |
| **Qwen3-30B-A3B** | | | | | |
| GAM | **62.50** | 43.24 | 62.31 | 76.00 | 61.01 |
| AIM | 42.81 | **52.70** | **84.62** | **84.00** | **66.03** |
| **Claude-4.5-Haiku** | | | | | |
| GAM | 76.56 | 81.08 | 88.46 | 80.00 | 81.53 |
| AIM | **84.69** | **94.59** | **93.08** | **95.00** | **91.84** |

# Summary: Memory Is the Foundation, Not a Feature

Agent memory enables agents to complete long-horizon tasks, such as Computer Use Agents, Coding Agent etc. It opens a door for many exciting applications:

In GDP eval [1], tasks requires memory

In personal assistants, tasks require memory

In multiagent systems, shared memory is important




[1] Patwardhan, Tejal, et al. "Gdpval: Evaluating ai model performance on real-world economically valuable tasks." *arXiv preprint arXiv:2510.04374* (2025).

# Attendance Question

Which of the following methods is not included in the slides:

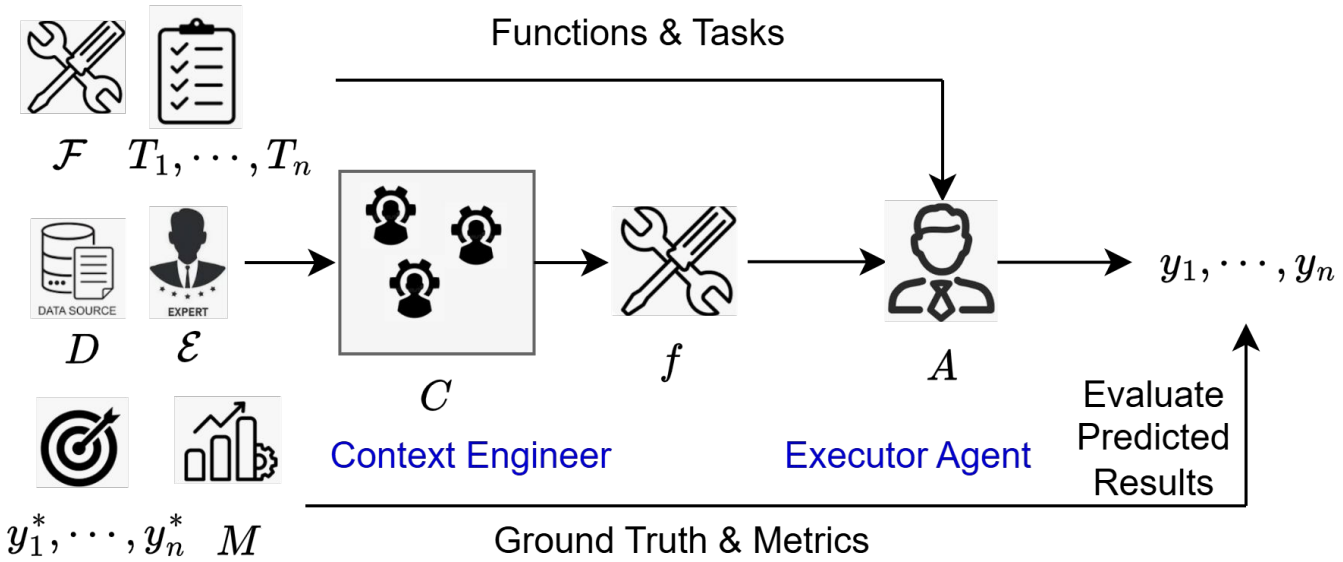
A. Mem0
B. A-Mem
C. Mem-Alpha
D. AIM

# Flat Structure
# RAG Dependent
# Predefined Structure

| Method | Information Extraction | Management Operations | Storage Structure | Retrieval Mechanism |
|---|---|---|---|---|
| A-MEM [93] | Direct archive, Summarization-based extract | Connect, Update | Flat, Vector | Vector-Based |
| MemoryBank [103] | Direct archive | Integrate, Update, Filter | Flat, Vector | Vector-Based |
| MemGPT [66] | Direct archive | Integrate, Transform, Update | Hierarchical, Vector | Lexical-Based, Vector-Based |
| Mem0 [11] | Direct archive, Summarization-based extract | Integrate, Update, Filter | Flat, Vector | Vector-Based |
| Mem0$^g$ [11] | Graph-based extract | Connect, Update, Filter | Flat, Graph | Vector-Based, Structure-Based |
| MemoChat [56] | Direct archive | Integrate | Flat | LLM-Assisted |
| Zep [72] | Direct archive, Graph-based extract | Connect, Transform, Update | Hierarchical, Graph | Lexical-Based, Vector-Based, Structure-Based |
| MemTree [73] | Direct archive | Connect, Integrate, Update | Flat, Tree | Vector-Based |
| MemoryOS [32] | Direct archive | Connect, Integrate, Transform, Update, Filter | Hierarchical, Vector | Lexical-Based, Vector-Based |
| MemOS [44] | Direct archive, Summarization-based extract | Connect, Integrate, Update | Hierarchical, Tree | Lexical-Based, Vector-Based |

# Problem Formulation

- Given a data source and optional high-level expert guidance, the goal is to build a context engineer that provides tools for the executor to query the context.



# Agent Memory

What is the problem of unstructured memory?

- Too choppy,
- RAG accuracy is low,
- Retrieve long document with higher token cost,
- Need to do multihop reasoning and filtering every time

I met Alice today, she is one of my best friend.

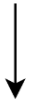
Alice, Bob, and me went to the park today. We are so happy!

Bob is annoying, I don't want to play with him anymore!

New turn:
You remmeber last time we went to the park? I started to feel we should still be good friends.

## Unstructured Memory

RAG Chunks about Park, Chunks about friends ~2000 tokens



Can you say more details? You go to the park very often and you have many friends.

## Structured Memory

SELECT * FROM Event WHERE Location = Park Order by Visit_Time

Okay, I understand this is a event that the user went to park with **Alice** and **Bob**. Let me search who they are

SELECT * FROM Friend WHEREname=Alice
SELECT * FROM Friend WHERE name = Bob

It seems Bob broke user's toy so that user doesn't like him..

I understand Bob did hurt your feeling, but he did no do it on purpose, he still wants to play with you. I think you should be good friends

# Agent Memory

Is there any existing database using SQL?

According to my search, no agentic system uses this.

https://docs.google.com/document/d/1xVvqDHtT4CJcQPSfONNXOJ0S7UQ94HE4gQydLVNjgQA/edit?usp=sharing

# Experiment - Target Question

*Paragraph A:* April Bernard

April Bernard (born 1956) is an American poet. She was born and raised in New England, and graduated from Harvard University. She has worked as a senior editor at "Vanity Fair", "Premiere", and "Manhattan, inc". In the early 1990s, she taught at Amherst College. In Fall 2003, she was Sidney Harman Writer-in-Residence at Baruch College. She currently teaches at Skidmore College in Saratoga Springs, New York. Her work has appeared in "The New Yorker", the "Boston Review", "AGNI", "Ploughshares", "Parnassus", and "The New York Review of Books".

*Paragraph B:* Premiere (magazine)

Premiere was an American and New York City-based film magazine published by Hachette Filipacchi Media U.S., between 1987 and 2010. The original version of the magazine, "Première", was established in France in 1976 and is still being published there.

**Q:** What are the publishing years of this American and New York based film magazine published by Hachette Filipacchi Media U.S., where April Bernard worked as senior editor?

**A:** between 1987 and 2010

# Experiment - Input User Prompt

Now I want you to store the dialogue memory in a DB schema, the information of the tables is shown as below:

```
CREATE TABLE People (
person_id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT UNIQUE,
birth_year INTEGER,
nationality TEXT,
occupation TEXT,
notes TEXT
);
```

```
CREATE TABLE Organization (
org_id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT UNIQUE,
type TEXT, -- e.g. 'Magazine', 'College', 'Publisher'
country TEXT,
location TEXT,
basic_info TEXT
);
```

```
CREATE TABLE Events (
event_id INTEGER PRIMARY KEY AUTOINCREMENT,
person_id INTEGER,
org_id INTEGER,
location_id INTEGER,
time_id INTEGER,
role TEXT,
description TEXT,
FOREIGN KEY (person_id) REFERENCES People(person_id),
FOREIGN KEY (org_id) REFERENCES Organization(org_id),
FOREIGN KEY (location_id) REFERENCES Location(location_id),
FOREIGN KEY (time_id) REFERENCES Time(time_id)
);
```

```
CREATE TABLE Time (
time_id INTEGER PRIMARY KEY AUTOINCREMENT,
description TEXT, -- e.g. "Early 1990s", "Fall 2003"
start_year INTEGER,
end_year INTEGER
);
```

```
CREATE TABLE Location (
location_id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT UNIQUE,
region TEXT,
country TEXT
);
```

Next, I will give you two paragraphs. For each sentence, you need to construct the entries and store them into the tables

# Experiment - Input Paragraph and Model Response

She was born and raised in New England, and graduated from Harvard University.

```
-- Table: Location
INSERT INTO Location (name, region, country)
VALUES ('New England', 'Northeastern United States', 'United States');


-- Table: Organization
INSERT INTO Organization (name, type, country, location, basic_info)
VALUES ('Harvard University', 'College', 'United States', 'Cambridge, Massachusetts', NULL);


-- Table: Events
-- Event 1: Born and raised in New England
INSERT INTO Events (person_id, location_id, role, description)
VALUES (1, 1, 'Childhood', 'Born and raised in New England');


-- Event 2: Graduated from Harvard University
INSERT INTO Events (person_id, org_id, role, description)
VALUES (1, 1, 'Student', 'Graduated from Harvard University');
```

# Experiment - Query and Final Answer

Q: What are the publishing years of this American and New York based film magazine published by Hachette Filipacchi Media U.S., where April Bernard worked as senior editor?

```
SELECT t.start_year, t.end_year
FROM Events e
JOIN Organization o ON e.org_id = o.org_id
JOIN Time t ON e.time_id = t.time_id
WHERE o.name = 'Premiere (American edition)';
```

Premiere (American edition) publishing years: 1987 - 2010

# Experiment - Analysis

=== Table: People ===

(1, 'April Bernard', 1956, 'American', 'Poet', None)

=== Table: Organization ===

(1, 'Harvard University', 'College', 'United States', 'Cambridge, Massachusetts', None)

(2, 'Vanity Fair', 'Magazine', 'United States', None, None)

(3, 'Premiere (American edition)', 'Magazine', 'United States', 'New York City', None)

=== Table: Location ===

(1, 'New England', 'Northeastern United States', 'United States')

(2, 'Saratoga Springs', 'New York', 'United States')

Takeaways:
LLMs can manage to work on existing schema and generate correct query to execute.

# Experiment - Dynamic Column

Change instruction: Now define a DB Schema to store the memory for LLMs. It contains several tables: Table name: People Organization Location Time Events Then, I will give some paragraphs and you need to define more columns dynamically

Paragraph: She was born and raised in New England, and graduated from Harvard University.

```
UPDATE People
SET birthplace = 'New England',
    education = 'Harvard University'
WHERE name = 'April Bernard';
```

Takeaways:

LLMs can manage to work on existing schema and generate correct query to execute. But cannot manage the columns or tables creation very well.

# Research Questions

RQ1: Can we design a database system specifically for Agent memory?

- Very ambitious. Require DB expertise, may work together with a DB person

RQ2: Can we use SQL to store Agent Memory?

- Schemas to design: Fixed schema, Agentic autonomy
- Facts to decompose and transfer to entities: Atom Content, Storage etc.

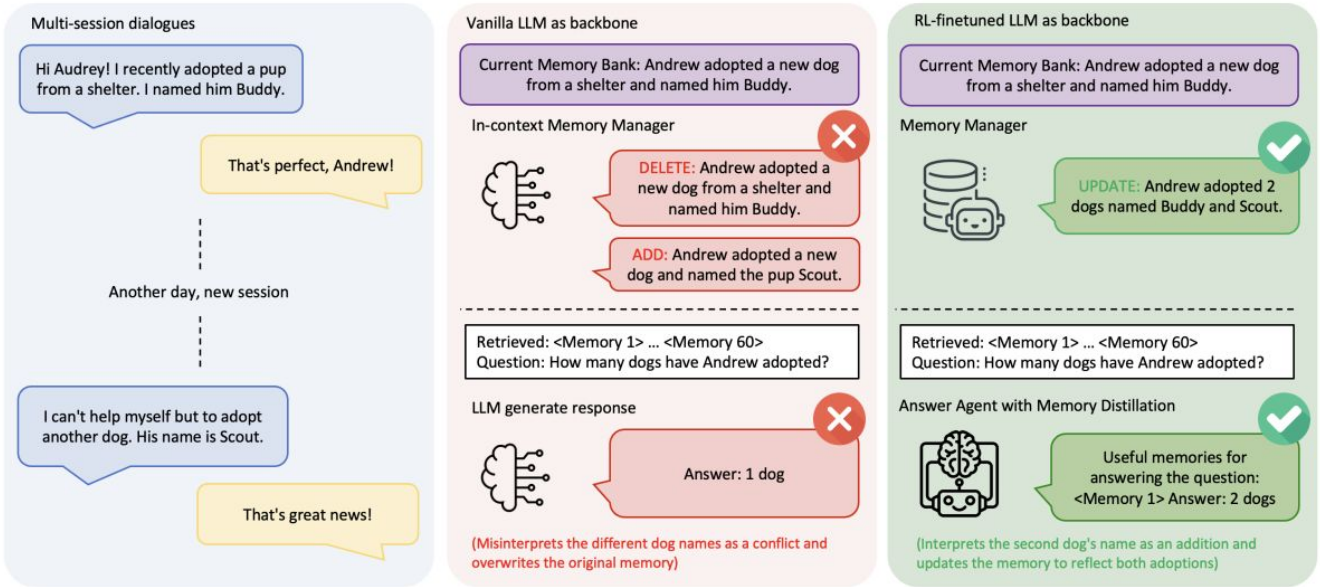
RQ3: Can we propose a benchmark for evaluation of Agent Memory?

- Real-user data, high-quality questions

RQ4: Can we train LLMs to dynamically understand when to query and what to query?

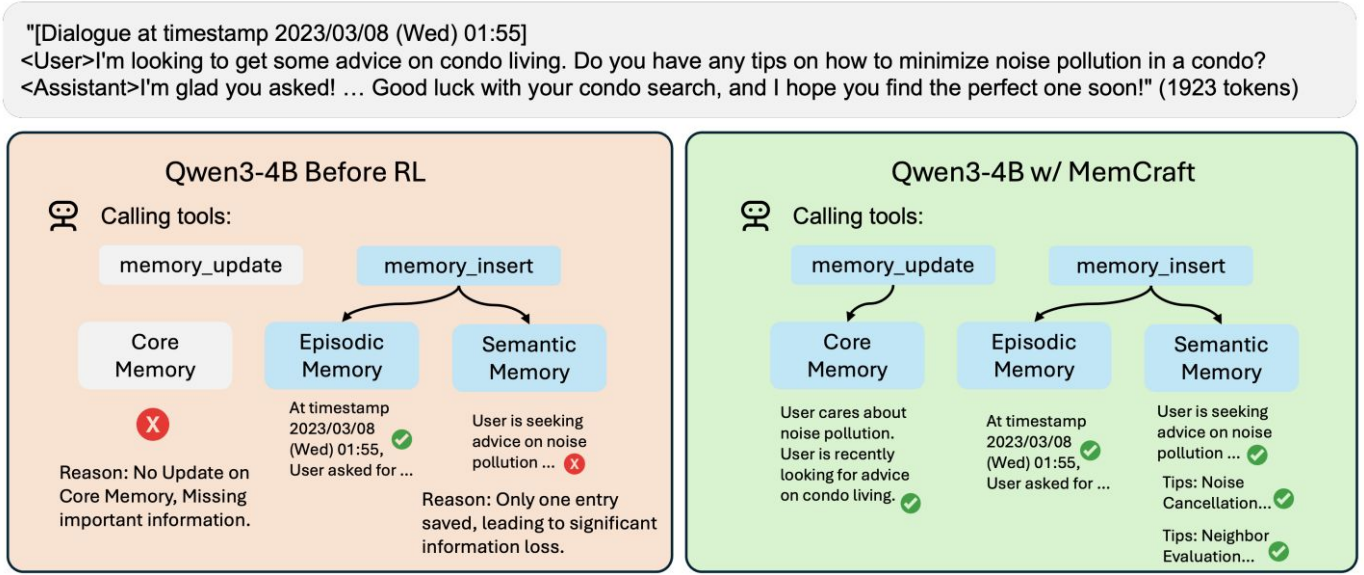
# How are the current Agent Memory designed?

Memory-R1: Enhancing Large Language Model Agents to Manage and Utilize Memories via Reinforcement Learning https://arxiv.org/pdf/2508.19828



# How are the current Agent Memory designed?

MEM-α: LEARNING MEMORY CONSTRUCTION VIA REINFORCEMENT LEARNING https://arxiv.org/pdf/2509.25911



# How are the current Agent Memory designed?

More work:

A-Mem: Agentic Memory for LLM Agents https://arxiv.org/pdf/2502.12110 They give us a good datasets/baselines setup

MIRIX: Multi-Agent Memory System for LLM-Based Agents https://arxiv.org/pdf/2507.07957 Multiagent Working memory!

Mem0: Building Production-Ready AI Agents with calable Long-Term Memory https://arxiv.org/pdf/2504.19413

# Outline

- Part 1. What Is Agent Memory?
- Part 2. How to Benchmark Agent Memory?
  - LoCoMo [1]
  - LongMemEval [2]
  - MemoryArena [3]
  - VibeCodingMem (ours)
- Part 3. How Are the Agent Memory Frameworks Designed?
  - A-Mem [4]
  - Mem0 [5]
  - LightMem [6]
  - AIM (ours)

[1] Maharana, Adyasha, et al. "Evaluating very long-term conversational memory of llm agents." *ACL 2024*.
[2] Wu, Di, et al. "Longmemeval: Benchmarking chat assistants on long-term interactive memory." *ICLR 2025*
[3] He, Zexue, et al. "MemoryArena: Benchmarking agent memory in interdependent multi-session agentic tasks." ArXiv Feb. 2026
[4] Xu, Wujiang, et al. "A-mem: Agentic memory for llm agents." *NeuRIPS 2025*
[5] Chhikara, Prateek, et al. "Mem0: Building production-ready ai agents with scalable long-term memory." *ArXiv Apr. 2025.*
[6] Fang, Jizhan, et al. "Lightmem: Lightweight and efficient memory-augmented generation." *ICLR 2026*